Vermilion: A Traffic-Aware Reconfigurable Optical Interconnect with Formal Throughput Guarantees

Vamsi Addanki
Purdue University

Deeksha Panduranga Rao
Purdue University

Goran Dario Knabe
TU Berlin

Chen Avin
Ben-Gurion University of the Negev

Stefan Schmid
TU Berlin

Abstract

The increasing gap between datacenter traffic volume and the capacity of electrical switches has driven the development of reconfigurable network designs utilizing optical circuit switching. Recent advancements, particularly those featuring periodic fixed-duration reconfigurations, have achieved practical end-to-end delays of just a few microseconds. However, current designs rely on multi-hop routing to enhance utilization, which can lead to a significant reduction in worst-case throughput and added overhead from congestion control and routing complexity. These factors pose significant operational challenges for the large-scale deployment of these technologies.

We present *Vermilion*, a reconfigurable optical interconnect that breaks the throughput barrier of existing periodic reconfigurable networks, without the need for multi-hop routing — thus eliminating congestion control and simplifying routing to direct communication. *Vermilion* adopts a traffic-aware approach while retaining the simplicity of periodic fixed-duration reconfigurations, similar to RotorNet. We formally establish throughput bounds for *Vermilion*, demonstrating that it achieves at least 33% more throughput in the worst-case compared to existing designs. The key innovation of *Vermilion* is its *short* traffic-aware periodic schedule, derived using a matrix rounding technique. This schedule is then combined with a traffic-oblivious periodic schedule to efficiently manage any residual traffic. Our evaluation results support our theoretical findings, revealing significant performance gains for datacenter workloads.

1 Introduction

Datacenters have experienced explosive growth in overall network traffic volume over the past decade [1]. With the recent introduction of high-bandwidth Machine Learning workloads into datacenters, the peak network traffic is expected to increase even more rapidly [2]. Unfortunately, traditional networks, which are built using electrical packet switches, struggle to keep up with this growing demand [3]. Further, the rapid evolution of datacenter applications and their changing bandwidth requirements implies: "the best laid plans quickly become outdated and inefficient, making incremental and adaptive evolution a necessity" [2]. This led to the emergence of novel technologies based on reconfigurable optical circuit switches [2–6]. Two prominent types of reconfigurable datacenter networks have emerged recently: traffic-oblivious [3, 4, 7, 8] and traffic-aware [5, 6, 9] networks. These networks are optically circuit-switched and feature bufferless switches. The circuits can be reconfigured, enabling the network topology to adapt dynamically to evolving communication patterns in datacenter workloads, which can potentially improve performance. The reconfiguration schedule varies between designs: some achieve high performance but are impractical for large-scale deployment, while others offer more practical solutions but at the cost of moderate performance.

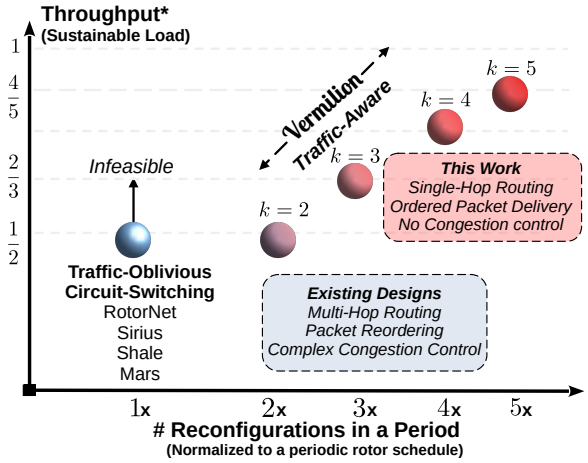


Figure 1: Existing designs based on periodic optical circuit-switching are oblivious to traffic patterns, requiring complex multi-hop routing and congestion control, which reduces throughput. *Vermilion* overcomes this limitation by introducing a few additional fixed-duration reconfigurations per period in a traffic-aware manner, while significantly simplifying both routing and congestion control.

From a performance standpoint, traffic-oblivious networks, such as RotorNet [4], Sirius [3], and Opera [10], offer low

reconfiguration overheads (in the range of nanoseconds) but sacrifice throughput due to their fixed and periodic switching schedules, which are independent of the underlying communication patterns. Only recently have the throughput bounds of traffic-oblivious networks been established [7, 11], showing that they are tightly bounded by $\frac{1}{2}$ i.e., a sustainable load of at most 50% under worst-case traffic patterns (e.g., ring communication), even with ideal routing and congestion control. In contrast, traffic-aware networks such as Mordia [12], Helios [6], and ProjecToR [5] are capable of achieving higher throughput because their switching schedules are optimized for the underlying communication patterns. Unfortunately, formal bounds on the achievable throughput of traffic-aware networks remain an open question in the literature. Nevertheless, with the evolution of distributed training paradigms such as Mixture-of-Experts (MoE), where communication demand exhibits a degree of predictability due to strong locality and load balancing across experts [13, 14], there is growing motivation to design traffic-aware network mechanisms that can effectively exploit this structure.

From a practicality standpoint, periodic fixed-duration reconfigurations have emerged as a promising design choice for reconfigurable datacenter networks [15, 16]. However, existing periodic networks are traffic-oblivious and require non-trivial additional support from hardware (NIC and switch) in terms of routing, packet reordering, congestion control and buffer architecture [3, 4, 8]. This complexity hinders the large-scale deployment of these networks in practice. In contrast, traffic-aware networks typically only require single-hop (direct) routing and do not depend on in-network congestion control mechanisms. However, this comes at the cost of complex reconfiguration schedules with variable durations, and they often rely on an additional packet-switched network [5, 6, 12].

In view of both performance and practicality, we explore a new direction in this paper: *Can a high-throughput network be designed using periodic fixed-duration reconfigurations, without relying on multi-hop routing?*

Figure 1 illustrates our perspective. Not only are existing designs limited in throughput, the worst-case throughput of *any* traffic-oblivious network is bounded by $\frac{1}{2}$ [7, 11, 17], making it infeasible to achieve higher throughput. However, it remains unexplored so far, whether and to what extent a traffic-aware approach to periodic reconfigurable networks can improve throughput. Intuitively, if the reconfiguration delay is negligible, a traffic-aware network can ideally achieve full-throughput. For instance, most prior works follow this intuition and use Birkhoff–von Neumann (BvN) [18] decomposition technique to devise a circuit switching schedule that perfectly matches the underlying traffic pattern [12, 19]. However, this not only results in a schedule with variable duration for each reconfiguration, it can also result in significantly low throughput due to reconfiguration overheads. Designing a traffic-aware network using only fixed-duration periodic reconfigurations, while surpassing the throughput limits of oblivious networks, requires new techniques and remains a challenging open problem.

We present *Vermilion*, a first traffic-aware reconfigurable network design that breaks the throughput barrier of existing designs, using only fixed-duration periodic reconfigurations. *Vermilion* not only achieves higher throughput but also greatly simplifies the protocol stack by eliminating multi-hop routing, congestion control and packet reordering. A recent work reports the following regarding single-hop routing in periodic circuit-switched network:

> *"Notably, none of this requires any modifications to the Linux application, TCP, or the Linux networking stack." [16]*

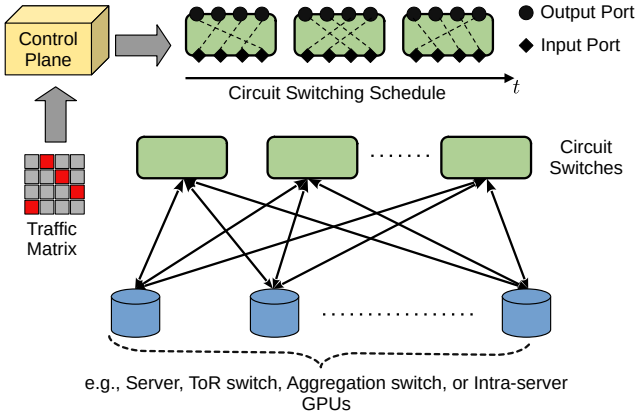
Vermilion relies solely on direct communication. As a result, *Vermilion* is more practical for deployment, within the available hardware capabilities e.g., using Rotor switches [16] and server-grade NICs. We discuss *Vermilion*'s practicality as well as future research directions in more detail later in this paper.

The key innovation behind *Vermilion* is the use of a matrix rounding technique [20] to derive a switching schedule that matches the underlying traffic pattern. This is in contrast to BvN decomposition technique [18] and greedy approximations [21] that have been largely used in the literature for designing traffic-aware networks [12, 19]. Importantly, our rounding technique allows decomposing a traffic matrix (after rounding) into a set of permutation matrices that directly serve as the switching schedule with *fixed-duration* for each circuit, in a periodic manner. Interestingly, the schedules produced by rounding technique are capable of serving a large portion of the demand but may leave certain residual demand. To this end, we introduce one extra cycle that is similar to that of existing designs, providing direct connections between all communicating nodes. Our rounding technique is simple and efficient to compute, making it practical for quick updates based on the changes in the traffic patterns. We establish the throughput bounds of *Vermilion*, marking the first theoretical result on the achievable throughput of traffic-aware networks, while accounting for reconfiguration delays.

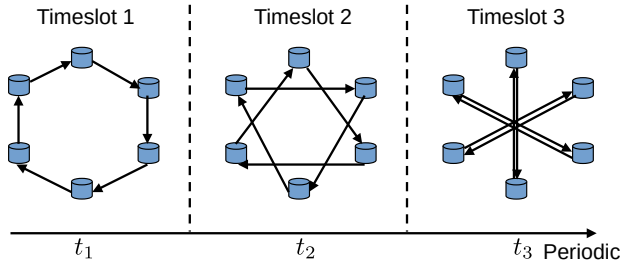
We evaluate using packet-level simulations and show that *Vermilion* significantly improves throughput by up to 2.13x and reduces the flow completion times by up to 99.54% compared to existing approaches. Furthermore, results show that *Vermilion* reduces the all-to-all completion time by up to 7.5x compared to BvN and by up to 24x Mixnet [22].

Our main contributions in this work are:

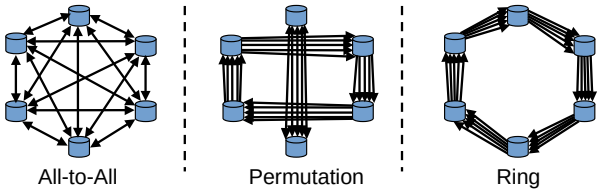
- A first separation result proving that traffic-aware reconfigurable datacenter networks are strictly superior to traffic-oblivious networks in terms of throughput.
- *Vermilion*, an innovative, yet simple, traffic-aware network design based on periodic fixed-duration reconfigurations. *Vermilion* achieves a throughput of at least $\frac{2}{3}$ (lower bound): a significant improvement over existing designs.



(a) The physical topology of a reconfigurable network consists of a set of nodes connected by optical circuit-switches arranged in a hierarchical Clos topology (e.g., leaf-spine). The circuit-switches are time-synchronized and rapidly reconfigure their circuits providing direct links between pairs of nodes in a periodic manner. We assume a control plane that defines the switching schedule for each switch.



(b) The physical topology establishes a node-to-node network that evolves over time periodically, allocating bandwidth between pairs of nodes in each timeslot.



(c) Examples of topologies that can be emulated over a *period* using periodic circuit-switching

Figure 2: The periodic circuit-switched network (2a) rapidly reconfigures its circuits, forming a dynamic node-to-node topology that evolves over time (2b). Over time, this topology emulates a specific static network (2c), for example, splitting bandwidth evenly across all pairs to create an all-to-all mesh topology (as in RotorNet [4] and Sirius [3]) or concentrating bandwidth between pairs with high demand to form a permutation or ring topology. The throughput of the network is heavily influenced by the choice of switching schedule, the emulated topology, and the underlying traffic matrix: *an optimization opportunity*.

- A simplification of the requirements from network protocol stack. *Vermilion* relies only on direct communication without multi-hop routing and does not need additional congestion control mechanisms to be deployed.
- Evaluations of *Vermilion* in both conventional traffic and distributed training workload, highlighting the performance benefits of *Vermilion* compared to traffic-oblivious counterparts in both the scenarios. Our results show that *Vermilion* significantly improves flow completion times for both short flows and long flows.
- All our artefacts and source code will be made available online together with this paper.

2 Motivation

In this section, we motivate our work by outlining the limitations of current periodic reconfigurable networks and the opportunities presented by adopting a traffic-aware approach within these systems. Our primary focus in the rest of this paper is on *periodic* reconfigurable networks, given the recent technological advancements showing their practicality and scalability [3, 16]. In the following, we briefly describe the network architecture that we consider in this paper.

Network model: Figure 2a illustrates the physical topology. A set of nodes are interconnected by optical circuit-switches such that at any time instance the network provides pair-wise direct connections across the nodes. For generality, we say "nodes" to refer to either servers, or ToR switches, or Aggregation switches, or intra-server components such as GPUs, that may be interconnected by an optical circuit-switched network. The optical circuit-switches reconfigure according to a schedule in a synchronized and periodic manner. Specifically, the circuit-switches reconfigure at *fixed-duration* intervals (timeslots) and each reconfiguration takes a specific amount of time (reconfiguration delay). This aligns with existing periodic circuit-switching technologies such as RotorNet [4, 15, 16] (Sirius [3]) with a reconfiguration delay of 7μs (3.84ns). As a result, the network topology evolves over time as shown in Figure 2b. In each timeslot, the degree of the topology is limited to the number of physical links. However, as the topology evolves over time, it can emulate a variety of topologies with high-degree, as shown in Figure 2c. For example, the topology can emulate an all-to-all mesh, a ring, or a permutation topology, depending on the periodic circuit-switching schedule. So far, in the literature, periodic reconfigurable networks were only studied in a traffic-oblivious setup i.e., the switching schedule is independent of the underlying traffic patterns, emulating an all-to-all mesh topology [3, 4, 10] or a d-regular topology [7, 8, 11] over time.

We first discuss our formal approach to optimizing the throughput of reconfigurable networks (§2.1), followed by

the drawbacks of existing traffic-oblivious designs (§2.2), and later, we make a case for traffic-aware networks (§2.3).

2.1 Throughput of Periodic Networks

Throughput offered by an interconnect is a crucial metric for assessing the sustainable load a network can handle, especially under highly concurrent communication patterns. In order to quantify the throughput, we first formally define the communication pattern i.e., the traffic matrix (Definition 1). The traffic matrix specifies the demand in bits per second between each pair of nodes i.e., the total demand originating from a source towards a destination. Following prior work [23], we consider the hose model [24] such that the total demand originating from (and destined to) each node is less than its corresponding capacity limits.

Definition 1 (Traffic matrix)**.** *Given a set of nodes $\mathcal{N}$, each with d outgoing and incoming links of capacity c, a traffic matrix specifies the traffic rate between every pair of nodes in bits per second defined as $\mathcal{M} = \{m_{u,v} \mid u \in \mathcal{N}, v \in \mathcal{N}\}$ where $m_{u,v}$ is the demand between the pair u,v. The traffic matrix is such that the total traffic originating at a source s is less than its outgoing capacity and the total demand terminating at a destination t is less than its incoming capacity i.e., $\sum_{u \in \mathcal{N}} m_{s,u} \leq c \cdot d$ and $\sum_{u \in \mathcal{N}} m_{u,t} \leq c \cdot d$.*

For a given communication pattern and the corresponding traffic matrix (Definition 1), we define throughput as the maximum scaling factor such that there exists a feasible flow that can satisfy the scaled demand subject to flow conservation and capacity constraints. We denote flow by $F : P \mapsto \mathbb{R}^+$, a map from the set of all paths P (static or temporal) to the set of non-negative real numbers. This mapping naturally ensures that the flow transmitted from a source eventually reaches the destination along a path $p \in P$. To obey capacity constraints, a feasible mapping is such that the sum of all flows traversing a link do not exceed the link capacity. We are now ready to define throughput formally.

Definition 2 (Throughput)**.** *Given a traffic matrix $\mathcal{M}$ and a reconfigurable network, throughput denoted by $\theta(\mathcal{M})$ is the highest scaling factor such that there exists a feasible flow for the scaled traffic matrix $\theta(\mathcal{M}) \cdot \mathcal{M}$. Throughput θ^* is the highest scaling factor for a worst-case traffic matrix i.e., $\theta^* = \min_{\mathcal{M} \in \hat{M}} \theta(\mathcal{M})$, where $\hat{M}$ is the set of all demand matrices.*

Intuitively, throughput for a specific communication pattern captures the maximum sustainable load by the underlying topology. Based on Definition 2, similar to prior works [7, 17, 23, 25], throughput of a topology is the minimum throughput across the set of all saturated demand matrices i.e., if a topology has throughput θ^*, then it can achieve at least throughput θ^* for *any* traffic matrix and at most throughput θ^* for a worst-case traffic matrix.

Several variants of the throughput problem have been studied over the last decades, especially in the context of the maximum concurrent flow problem [26]. However, in contrast to static networks, the fundamental challenge to study throughput in the context of reconfigurable networks is that the topology changes over time and can even be a function of the traffic matrix in the case of traffic-aware networks.

In the specific case of periodic networks, a recent work establishes an equivalence between the throughput of any periodic network and a corresponding emulated topology (as static graph), allowing the study of throughput in periodic networks using existing techniques [7]. We present a formal definition of emulated graph in Appendix B. Essentially, the emulated graph is a time-collapsed view over an entire period of the periodic network as shown in Figure 2c.

For example, in RotorNet, the length of the period is $\Gamma = \frac{n}{d}$ timeslots. If a link appears once between every pair of nodes over a period, then the emulated graph has one link between every pair of nodes (complete graph), where each link has a capacity of $c \cdot \frac{1}{\Gamma} = c \cdot \frac{d}{n}$. As we will discuss later, emulating a complete graph results in a drop in throughput by a factor 2 under any permutation (e.g., ring) communication patterns. Intuitively, in order to maximize throughput, the emulated graph must provide high bandwidth between specific pairs of nodes with high-demand i.e., by emulating a topology that closely matches the underlying communication pattern.

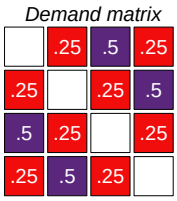
■ ***Takeaway.*** *Optimizing periodic networks entails finding the best static graph that can be emulated over a period, that provides the highest throughput for an (any) underlying communication pattern.*

2.2 Drawbacks of ORNs

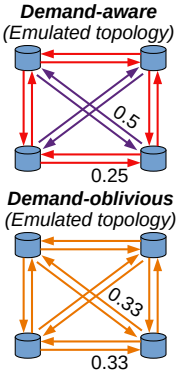
Oblivious periodic circuit-switching has an obvious advantage of simplicity without any control plane involvement for optimizing the topology. However, this leads to certain drawbacks that we elaborate in the following.

Factor of 2 reduction in throughput: Recent works have established a tight bound of $\frac{1}{2}$ on the throughput of periodic networks [7, 11, 17]. We emphasize that this reduction in throughput is not solely attributed to the use of valiant load-balancing; rather, the network fundamentally cannot sustain beyond 50% load, even with an ideal routing scheme, under a worst-case communication pattern such as a ring. Specifically, in ring communication, only specific node pairs exchange data, but emulating a complete graph only offers a capacity of $c \cdot \frac{d}{n}$ (as discussed above), falling short of the ideal bandwidth provisioning of $c \cdot d$ (leveraging all available links) for these pairs. Even an optimal routing scheme is thus forced to route traffic over 2-hop paths, in order to fully utilize the network capacity, leading to a throughput of $\frac{1}{2}$.

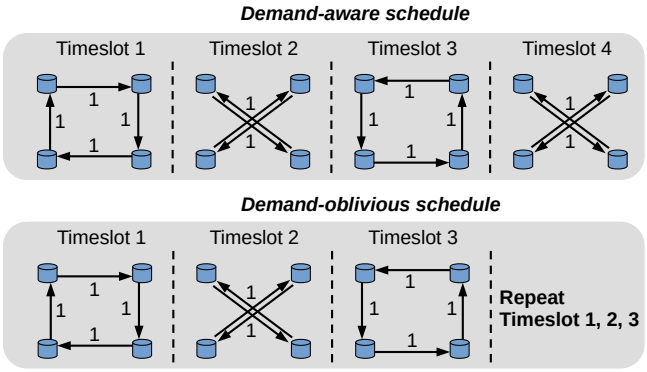
Multi-hop routing: The use of indirect paths is a necessity to improve utilization in oblivious periodic networks. However, since the network is evolving over time, this implies that intermediate nodes need to buffer (hold) traffic until the next link along the path becomes available. This results in



(a) Example traffic matrix for a topology with 4 nodes, each with 1 physical link.



(b) Target emulated topology for traffic-aware vs oblivious periodic networks.



(c) Derived periodic schedule for a traffic-aware vs oblivious periodic network.

Figure 3: Interpreting the traffic matrix (Figure 3a) as a target emulated graph (Figure 3b) allows deriving a periodic schedule (Figure 3c) that can achieve full throughput for traffic-aware periodic networks, even with single-hop routing. In contrast, the schedule for an oblivious periodic network often does not match the underlying traffic matrix, necessitating multi-hop routing and resulting in reduced throughput.

additional latency and buffer requirements. In addition, the use of multi-hop routing requires the use of in-network congestion control mechanisms to prevent packet loss due to buffer overflows [8]. This further complicates the network stack and can lead to performance degradation under high load. In fact, recent works even suggest sacrificing throughput further in order to satisfy buffer constraints [7].

Packet reordering: Not only do existing oblivious networks require multi-hop routing, but they also require multi-path transmission in order to fully utilize the network. This results in packets arriving out-of-order at the destination, necessitating additional support such as reorder buffers [16]. This can be particularly challenging in the context of RDMA since RoCE implementations typically react to packet reordering at the receiver with NACK that triggers retransmissions immediately at the sender.

■ ***Takeaway.*** *While oblivious periodic networks simplify switching schedule selection, they rely on complex protocols such as multi-hop routing, multi-path transmission, and reorder buffers, which can significantly reduce the maximum achievable throughput.*

2.3 A Case for Traffic-Aware Networks

In contrast to oblivious reconfigurable networks, traffic-aware networks can potentially achieve higher throughput by optimizing the network topology for the underlying communication pattern. traffic-aware networks have been studied in the literature [5, 6, 12] and have been empirically shown to achieve better performance compared to oblivious networks. In fact, traffic-aware networks can satisfy any demand within the hose model, if the reconfiguration delay is hypothetically near-zero or negligible. This result has been intuitively known in the literature [12, 27], which relies on Birkhoff-von-Neumann (BvN) matrix decomposition technique [18]. For completeness, we formally state it here with a proof in Appendix B.

Theorem 1 (Ideal throughput of traffic-aware network)**.** *The throughput of an ideal traffic-aware reconfigurable network is* 1 *i.e., full-throughput for any traffic matrix if the reconfiguration delay is negligible.*

The core intuition behind BvN-based traffic-aware network design is to allocate bandwidth between node-pairs as a convex combination (over time) of permutations derived from BvN matrix decomposition. However, this approach suffers from two key limitations: *(i)*, the durations between reconfigurations in the switching schedule are variable, and *(ii)*, the time between reconfigurations can be shorter than the reconfiguration delay, leading to a significant drop in throughput. Consequently, BvN-based designs are incompatible with current technologies that rely on fixed-duration periodic reconfigurations.

Interestingly, it is possible to design periodic reconfigurable networks that achieve full throughput for specific types of communication patterns. For instance, consider a ring traffic matrix. Even with fixed-duration periodic reconfigurations, a simple switching schedule with a period of one timeslot — where the direct links between communicating nodes in the ring are maintained — can achieve full throughput. In contrast, oblivious periodic networks can only achieve a throughput of $\frac{1}{2}$ for the same communication pattern. Similarly, if the traffic matrix consists of integer multiples of link capacity, it is straightforward to design a switching schedule that achieves almost full throughput. Simply establishing direct links between communicating nodes is sufficient to achieve full throughput for such traffic matrices.

Our key insight is that when the periodic schedule is constrained to Γ timeslots, any traffic matrix $\mathcal{M}$ with non-zero demand represented as an integer multiple of $\frac{c}{\Gamma}$ can be

satisfied with full throughput. For such communication patterns, a feasible periodic schedule with fixed-duration reconfigurations always exists and achieves full throughput. We formally prove this result in Appendix B. Intuitively, the traffic matrix can be visualized as an edge-weighted graph, where the weights represent the demand between node pairs. This graph corresponds to the target emulated topology for the periodic schedule, with weights now representing link capacities. A periodic schedule can then be derived to allocate bandwidth between source-destination pairs according to the traffic matrix, ensuring full throughput.

Theorem 2 (Throughput under integer traffic matrices)**.** *There exists a periodic reconfigurable network with a period of* Γ *timeslots with* Δ_r *fraction of time spent in reconfiguration, that can achieve nearly full throughput of* $1-\Delta_r$ *using only single-hop routing for traffic matrices where any non-zero demand is an integer multiple of* $c \cdot \frac{1}{\Gamma}$*, where* c *is the link capacity in the physical topology.*

Figure 3 illustrates a traffic matrix for which a traffic-aware periodic schedule can be trivially derived. The traffic matrix consists of integer multiples of $\frac{1}{4}$, and the corresponding switching schedule, with a length of 4 timeslots, is depicted in Figure 3. This schedule enables the network to fully satisfy the traffic matrix within each period, achieving full throughput. In contrast, the schedule of an oblivious network contains only 3 timeslots, omitting the additional 4th timeslot used in the traffic-aware schedule. The oblivious schedule distributes uniform bandwidth across all node-pairs but mismatches the underlying traffic matrix, as demonstrated in Figure 3. Notably, the inclusion of just a few extra timeslots in a traffic-aware manner can significantly enhance the network's throughput.

Single-hop routing: Notice that the traffic-aware schedule in Figure 3 achieves high throughput even with single-hop routing. This is because the emulated topology provides capacity between each node pair that precisely matches the traffic matrix specifications. Generally, any schedule derived from Theorem 2 maintains this single-hop routing advantage, simplifying the protocol stack by eliminating the need for multi-hop routing, congestion control, and packet reordering.

However, a caveat is that increasing the period Γ can lead to an excessively long periodic schedule to achieve full throughput for any traffic matrix, as indicated by Theorem 2. This can result in unacceptable delays. Theorem 2 suggests potential throughput gains by deriving periodic schedules tailored to the underlying communication patterns. The challenge remains in deriving *short* schedules while still achieving high throughput.

■ ***Takeaway.*** *Specific communication patterns showcase the substantial throughput gains that traffic-aware periodic networks can achieve over oblivious designs. The main challenge — and opportunity — lies in deriving compact schedules that can achieve high throughput for any communication pattern.*

3 Traffic-Aware Periodic Network

Based on our observations in §2, we seek to design a *simple* traffic-aware periodic network, within the practical capabilities of existing optical circuit-switching technologies [3, 15, 16] and the end-host networking stack. Our goal is to achieve high throughput for any traffic matrix using only single-hop routing i.e., direct communication. We first present our network design (§3.1), followed by the throughput guarantees of our design (§3.2) and its practicality (§3.3).

3.1 Vermilion

We present *Vermilion*, a first traffic-aware periodic network design that can probably achieve high throughput compared to existing oblivious designs. We walk through each component of *Vermilion* in the following.

Physical topology: Our network model remains the same as described in §2, with a set of nodes interconnected by optical circuit-switches in a hierarchical CLOS topology. The circuit-switches are synchronized in time and reconfigure at fixed-duration intervals, forming a dynamic node-to-node topology that evolves over time. Each node in the topology has $\hat{d}$ physical links that connect to the optical interconnect, hence at any time instance, each node can connect to at most $\hat{d}$ other nodes. The physical links have a capacity of c.

Parameters: *Vermilion* has one parameter k, that controls the degree of the target emulated topology. A higher k leads to higher throughput but also increases the schedule length. The resulting schedule can achieve at least $\frac{k-1}{k}$ throughput for a given traffic matrix $\mathcal{M}$. For example, even with $k=3$, *Vermilion* can achieve a throughput of $\frac{2}{3}$, breaking the throughput bounds of oblivious periodic networks. We discuss the choice of k in §3.2.

Periodic schedule: The key innovation in *Vermilion* is its traffic-aware periodic schedule. Algorithm 1 outlines the steps to derive the periodic schedule. Figure 4 shows an example workflow of *Vermilion* for $k = 3$. Given a traffic matrix $\mathcal{M}$, we first generate an emulated topology that can achieve high throughput for the given traffic matrix. Our emulated topology is always regular and allows us to then decompose it into a periodic schedule. We construct the emulated topology as follows:

- **Matrix rounding:** We first normalize the traffic matrix such that the maximum sum of any row and column is at most 1. We then upscale the traffic matrix by $(k-1)\cdot n$, where k is a parameter to *Vermilion* and n is the number of nodes. We round entries of the scaled matrix such that the sum of each row and column remains the same, a technique known as matrix rounding [20].

- **Traffic-aware multigraph:** Based on the rounded matrix, we construct a multigraph by adding edges between each node pair based on the rounded matrix. For instance if the rounded matrix specifies 2 between s-t node pair, then we add 2 edges between s and t.

Algorithm 1: *Vermilion*

Input : Traffic matrix $\mathcal{M}$, number of nodes n, degree $\hat{d}$, link capacity c, parameter k

procedure GENERATESCHEDULE($\mathcal{M}$):
  G = emulatedTopology($\mathcal{M}$)
  ▷ *Sequence of matchings for the periodic schedule*
  for $i = 1$ **to** $k \cdot n$ **do**
    remove one perfect matching M_i in G
    add M_i to schedule
  end for
  return M
procedure EMULATEDTOPOLOGY($\mathcal{M}$):
  ▷ *Initialize a multigraph*
  $G = (V,E), V = \{1,...,n\}, E : V \times V \to \mathbb{N}$
  Normalize the traffic matrix $\mathcal{M}$
  $\mathcal{M} \leftarrow (k-1) \cdot n \cdot \mathcal{M}$
  $\mathcal{R}$ = Round($\mathcal{M}$) ▷ *Matrix rounding*
  for *each node pair* (u,v) **do**
    ▷ *Allocate bandwidth for bulk demand*
    $E((u,v)) \leftarrow \mathcal{R}(u,v)$ ▷ *# Edges between u,v*
    ▷ *Allocate bandwidth for residual demand*
    $E((u,v)) \leftarrow E((u,v)) + 1$
  end for
  ▷ *Ensure that the final graph is regular*
  $x^{in} \leftarrow k \cdot n - x^{in}$ ▷ *Remaining in-degree*
  $x^{out} \leftarrow k \cdot n - x^{out}$ ▷ *Remaining out-degree*
  $G' = (V,E')$ = ConfigurationModel(x^{in}, x^{out})
  $E \leftarrow E \uplus E'$
  return G

This ensures that majority of the traffic matrix is served efficiently in a traffic-aware manner.

- **Traffic-oblivious residual graph:** Matrix rounding may not exactly match the original traffic matrix and can leave some residual demand. To address this, we ensure that any residual demand can be routed by adding one additional edge between each node pair in the multigraph. This step guarantees any-to-any connectivity and is traffic-oblivious, meaning it is independent of the specific traffic matrix.

- **Augmenting the regularity of the graph:** At this point, say each node has in-degree x_i^{in} and out-degree x_i^{out}, we then take the degree sequences $\langle k \cdot n - x_1^{in}, ..., k \cdot n - x_n^{in} \rangle$ and $\langle k \cdot n - x_1^{out}, ..., k \cdot n - x_n^{out} \rangle$, and add additional links to our graph based on configuration model using the above degree sequence.

The resulting graph is our target emulated topology. The above construction always leads to a multigraph with degree $k \cdot n$, a directed regular graph, that can be decomposed into $k \cdot n$ number of perfect matchings. These perfect matchings are then executed in round-robin periodically, using optical circuit switching.

Routing: *Vermilion* relies solely on single-hop routing, as the topology provides sufficient direct links between communicating nodes over time, ensuring high throughput even with direct communication.

Congestion control: *Vermilion* does not explicitly require any congestion control algorithm in the network. If the network is all-optical i.e., when servers connect directly to circuit switches, then *Vermilion* does not require even end-host congestion control (except for reliability) since every packet is transmitted directly to the destination. The direct communication paths ensure that the network is not congested.

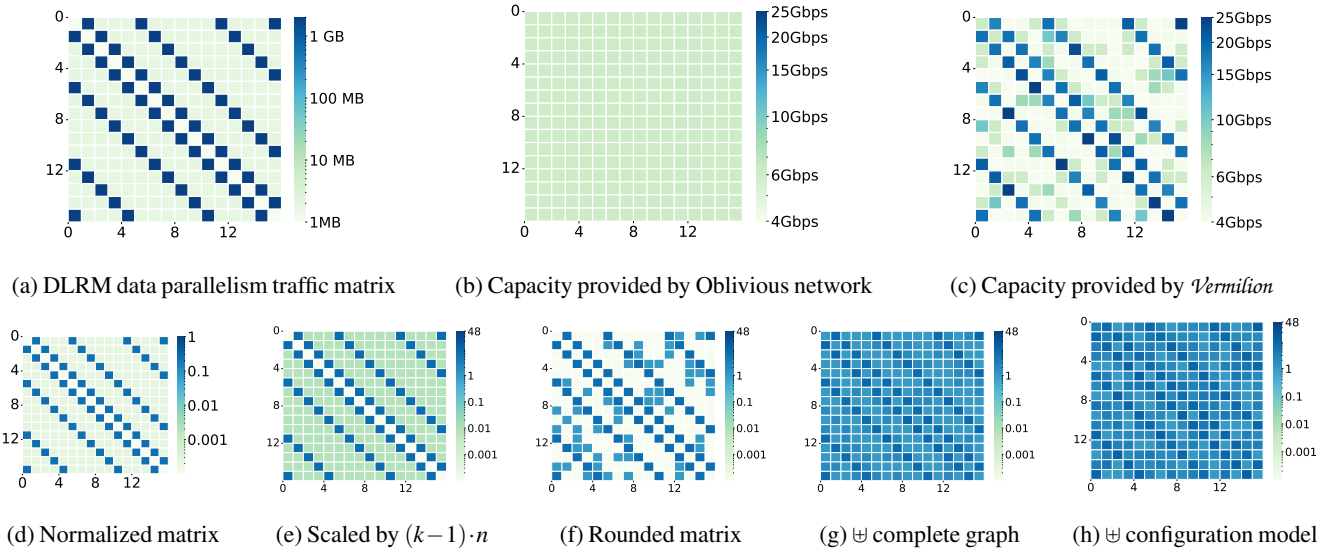
Flow scheduling: Given that packets reach destinations directly, flow scheduling dominates in determining per-flow performance. We consider that the packets of all active flows at the end-host are scheduled in a round-robin manner and all transmissions are paused during the synchronized reconfiguration events, similar to prior work [16]. Scheduling algorithms such as shortest remaining processing time (SRPT) based on remaining flow size could potentially improve flow completion times. We leave the design of tailored scheduling algorithms for future work.

3.2 Throughput Guarantees of Vermilion

Vermilion offers attractive theoretical guarantees on throughput and consequently completion time for a given traffic matrix. We discuss the key properties of *Vermilion* in the following.

Theorem 3 (Throughput lower bound)**.** *Vermilion achieves a throughput of at least* $\frac{k-1}{k} \cdot (1 - \Delta_r)$ *using single-hop routing for any given traffic matrix within the hose model, where* Δ_r *is the fraction of time spent in reconfiguration and* k *is a parameter to Vermilion.*

Proof. We assume, without loss of generality, that the capacity of each physical link is 1 and the degree of the physical topology is $\hat{d}$. A traffic matrix $\mathcal{M}$ in the hose model has the property that the sum of every row and column is at most $\hat{d}$. Let the period (number of timeslots) of the periodic schedule be $k \cdot \frac{n}{\hat{d}}$, i.e., k times the period of an oblivious periodic schedule. The emulated multigraph G has a degree of $k \cdot n$, and each link has capacity $\frac{1}{k \cdot \frac{n}{\hat{d}}} = \frac{\hat{d}}{k \cdot n}$. We now upscale both the traffic matrix $\mathcal{M}$ and each link capacity of the emulated multigraph by $\frac{k \cdot n}{\hat{d}}$. Thus, it is equivalent to find the throughput of the emulated multigraph G with degree $k \cdot n$ and each link having capacity 1, under the scaled traffic matrix $\mathcal{M}' = \frac{k \cdot n}{\hat{d}} \cdot \mathcal{M}$, where the sum of each row and column is at most $k \cdot n$. We seek to find the edge multiset of the emulated multigraph G that maximizes throughput. In order to show that *Vermilion* achieves a throughput of at least $\frac{k-1}{k}$, it suffices to show that *Vermilion* can satisfy $\frac{k-1}{k} \cdot \mathcal{M}'$ within the capacity constraints. According to *Vermilion*, edges are added based on matrix rounding using $\frac{(k-1) \cdot n}{\hat{d}} \mathcal{M} = \frac{k-1}{k} \cdot \mathcal{M}'$. These edges satisfy all the demand in



(a) DLRM data parallelism traffic matrix (b) Capacity provided by Oblivious network (c) Capacity provided by *Vermilion*

(d) Normalized matrix (e) Scaled by $(k-1)\cdot n$ (f) Rounded matrix (g) $\uplus$ complete graph (h) $\uplus$ configuration model

Figure 4: An example workflow of *Vermilion* with (4a) DLRM data parallel traffic matrix. An oblivious periodic network (4b) provides uniform capacity between all node pairs irrespective of the traffic matrix. *Vermilion* (4c) provides capacity between node pairs based on the underlying traffic matrix. *Vermilion* first (4d) normalizes the given traffic matrix, upscales it by $(k-1)\cdot n$ (4e), rounds it (4f), augments it with a complete graph (4g) for any-to-any connectivity, and finally augments it with additional links to ensure regularity (4h). The resulting matrix (a multigraph) is the target emulated topology (4c), which is then decomposed into a sequence of matchings for the periodic schedule.

the traffic matrix $\frac{k-1}{k}\cdot\mathcal{M}'$ except for entries that were rounded down during the rounding process. This is because, the sum of every row and column in the traffic matrix (and the rounded matrix) is at most $(k-1)\cdot n$, entries are either rounder up or down, and links are added only to non-zero entries. The rounding process utilizes at most $(k-1)\cdot n$ incoming and outgoing links from each node. We are left with at least n incoming and outgoing links for each node. The residual demand is then served by adding additional edges between each node pair. The residual demand between any node pair is strictly less than 1 (due to rounding), and a single additional link between the pair can fully satisfy the residual demand. Finally, each link loses $(1-\Delta_r)$ fraction of capacity due to reconfigurations and hence the overall throughput is $\frac{k-1}{k}\cdot(1-\Delta_r)$. □

Theorem 3 suggests that a throughput of $\frac{k-1}{k}$ is achievable for any given traffic matrix, with k acting as a control parameter for throughput. Essentially, k represents the factor by which the schedule of *Vermilion* is elongated compared to an oblivious schedule that provides periodic any-to-any connectivity. For instance, with $k=3$, *Vermilion* guarantees a throughput of $\frac{2}{3}$, and this can be further increased by increasing k, resulting in higher throughput. However, this comes at the cost of longer schedules and increased delay. As a result, the throughput guarantee can only be achieved over extended periods of time, making higher k values potentially unsuitable for workloads with stringent latency requirements. In our evaluations, we use $k=3$ by default, as it strikes a good balance between throughput and schedule length.

3.3 Practicality of Vermilion

We discuss the practicality of *Vermilion* in the context of modern datacenter infrastructure and optical circuit-switching technologies. A detailed discussion appears in Appendix A.

Scalability: Periodic circuit-switched networks in general exhibit excellent scalability properties [3, 4, 8, 15]. The required switch size is arguably the most important scalability concern in these networks. A simple leaf-spine topology may quickly become limited in terms of the required switch size. *Vermilion* can scale to large topologies with thousands of nodes interconnected by circuit switches arranged in a non-oversubscribed k-ary fattree topology [28]. Fattree allows any permutation to be executed. For example, even a greedy algorithm can find edge-disjoint paths corresponding to a required matching. These edge-disjoint paths then reveal the required circuits at each switch in the network, consequently, the sequence of matchings for each switch can be obtained. Further, *Vermilion* is much more scalable than oblivious periodic networks in terms of protocol stack as it only requires single-hop routing and does not rely on complex in-network congestion control algorithms.

Complexity: Deriving a schedule based on *Vermilion* is solvable in polynomial time. Specifically, matrix rounding is polynomial time solvable [20], and all other transformations have a complexity of $O(n^2)$, which is inherent to any traffic-aware approach due to the need to traverse the traffic matrix at least once. We present the absolute times required to compute *Vermilion*'s schedule in Appendix A. In contrast, approaches that rely on Birkhoff decomposition not only produce schedules with

variable reconfiguration durations but also face the challenge that finding a schedule of minimum length using Birkhoff decomposition is known to be NP-hard [29]. We further discuss the challenges of deriving fixed-duration schedules using Birkhoff decomposition and time quantization in Appendix A.

Updating the schedule: *Vermilion* being a traffic-aware design, it requires that the switches can be updated with a new periodic schedule when the communication patterns change. Efficiently updating the circuit-switching schedule is an active area of research, particularly for fast-reconfigurable periodic circuit switches operating on microsecond (μs) or nanosecond (ns) timescales [16, 30]. We present a detailed discussion on the various choices for switching fabrics in Appendix A. Commercially available optical switches that can reconfigure at millisecond scale already allow for updating the switch with arbitrary matchings via control plane [31].

Traffic matrix estimation: Modern datacenters are capable of accurately estimating the traffic matrix at scale [2]. Further, more recent distributed training workloads in GPU clusters have a predictable traffic matrix that is also periodic in nature [32–34]. All-to-all communication in Mixture-of-expert workloads is predictable before the dispatch phases [22]. In Appendix A, we describe our approach for estimating the traffic matrix in a fully distributed manner when the workload is unpredictable, leveraging the oblivious phase of *Vermilion*'s schedule to perform an allGather operation to estimate the traffic matrix.

Overall, we believe *Vermilion* does not fundamentally require novel hardware components, and is well within the practical capabilities of existing technologies. We discuss the limitations of our current design and potential future directions in Appendix D.

4 Evaluation

We evaluate the performance of *Vermilion* in terms of flow completion times and link utilization using real-world datacenter workloads (§4.1), in terms of all-to-all completion times using Mixture-of-expert workloads (§4.2), and in terms of throughput with commonly observed demand patterns in datacenters (§4.3). We compare *Vermilion* with existing periodic network designs, namely, RotorNet [15], Opera [10], Sirius [3], and traffic-aware greedy matching, MixNet [22], and BvN [18] schedules. Due to space constraints, we present our additional results in Appendix E

4.1 Flow Completion Times & Link Utilization

Our evaluation in this section is based on packet-level simulations using htsim [35].

Topology: We consider a datacenter consisting of 512 servers arranged across 64 top-of-rack (ToR) switches. These ToR switches are interconnected by a layer of 8 optical circuit switches, with all link capacities set to 100Gbps. For all systems compared, including *Vermilion*, the circuit switches have a reconfiguration delay of $0.5\mu s$, which is the best-case switching time for the latest version of RotorNet [16].

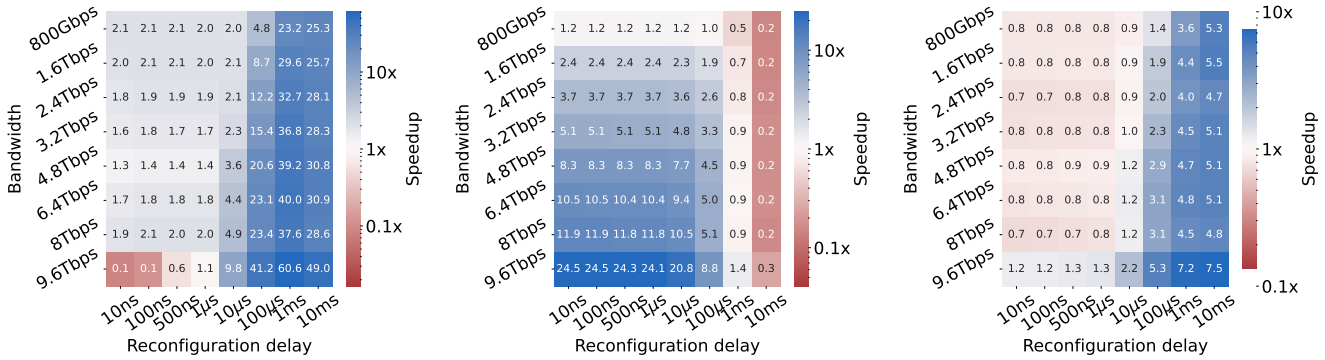
Comparisons & configurations: We set $k = 3$ for *Vermilion*, and compare it with RotorNet [4], Opera [10] and Sirius [3], representing traffic-oblivious approaches. We also compare *Vermilion* with a traffic-aware approach that adapts its switching using maximum weight matching based on the underlying traffic matrix, similar to Negotiator [36]. We refer to this baseline as Greedy Matching. For all systems, we set the slot time to $9\times$ the reconfiguration delay. Opera, on the other hand, internally determines its slot time based on the propagation delay [10]. We use the recommended configurations of these systems as provided in their respective papers. Specifically, RotorNet uses RotorLB load-balancing algorithm for managing congestion between the ToRs; Opera uses k-shortest paths for short flows within the same timeslot to reduce FCTs; Sirius uses valiant load balancing (VLB) [3, 37]; and *Vermilion* uses single-hop routing without any further congestion control mechanisms. All systems use NDP [38] as the transport protocol. In the case of *Vermilion*, we turn off all actions of NDP and set a constant congestion window size.

Workload: We launch the websearch [39] workload, a widely-used datacenter benchmark from prior work. We simulate a pair-wise communication pattern between racks and vary the load between 5% and 60% of the server link capacity. Flows arrive according to a poisson process such that an average load is achieved on each server's outgoing link. We construct a periodic schedule for *Vermilion* based on the average load[1], while the other systems use an oblivious schedule (emulating an all-to-all mesh topology) [3, 4, 10] for all loads. Greedy Matching baseline determines its topology based on the maximum weight matching of the underlying traffic matrix. We report the 99-percentile flow completion times (FCTs) and the average link utilization across server downlinks.

***Vermilion* significantly improves short flow FCTs:** Figure 12 shows the flow completion times for the websearch workload. *Vermilion* significantly improves the 99-percentile FCTs compared to existing designs, even at low loads. Figure 12a, at 5% load, shows that *Vermilion* improves the 99-percentile FCTs by 99.28% compared to RotorNet, by 51.38% compared to Opera, by 98.2% compared to VLB, and by 95.5% compared to Greedy Matching. This is because *Vermilion* provides direct communication links between communicating pairs, reducing the FCTs for short flows. As the load increases, *Vermilion* improves the FCTs for short flows on average by 99.54% compared to RotorNet, by 61.5% compared to Opera, by 90.96% compared to VLB, and by 81.57% compared to Greedy Matching.

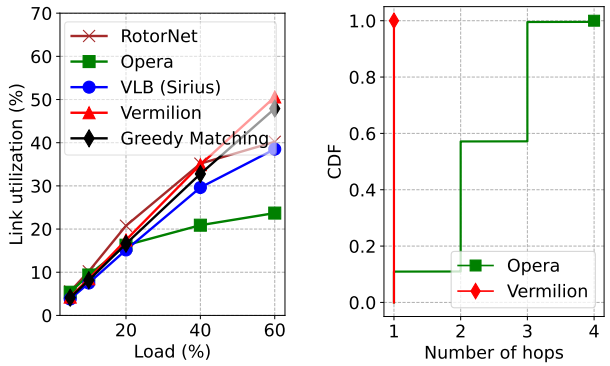
***Vermilion* does not compromise long flow FCTs:** While *Vermilion* significantly reduces short flow FCTs, it also improves the FCTs for long flows. Figure 12 shows that *Vermilion*

[1]The instantaneous traffic matrix is in fact variable but we optimize based on the average load.



(a) VLB (b) MixNet (c) BvN

Figure 5: All-to-All completion time speedup ($\frac{Alg}{Vermilion}$) for Deepseek MoE workload across different reconfiguration delays and optical fabric link bandwidths.



(a) Link utilization (b) Path lengths

Figure 6: [Websearch workload] Opera uses k-shortest paths for short flows, leading to longer path lengths and loss in link utilization, whereas *Vermilion* uses exclusively direct paths and achieves higher link utilization compared to other designs.

outperforms alternative approaches in terms of long flow FCTs. This is primarily due to *Vermilion*'s traffic-aware schedule, which provides direct links between communicating pairs. Across various loads (Figure 12), *Vermilion* achieves comparable FCTs to other systems. *Vermilion*'s high-throughput capability is especially beneficial for long flows, as they are bandwidth-intensive and require a robust interconnect to maintain low FCTs, while not penalizing short flows. In contrast, RotorNet achieves better FCTs for long flows, but at the cost of significantly higher FCTs for short flows.

***Vermilion* improves average link utilization:** We sample the link utilization of each server at 10μs intervals and report the average link utilization across all downlinks in the network. Figure 6 highlights significant improvements in link utilization with *Vermilion*. Up to 10% load, *Vermilion* achieves similar link utilization to RotorNet and Opera. However, as the load increases, Opera's utilization saturates at $\approx$ 23%. At 60% load, *Vermilion* improves average link utilization by 1.25$\times$ compared to RotorNet, 2.13$\times$ compared to Opera, 1.31$\times$ compared to VLB, and by 1.05$\times$ compared to Greedy Matching. Figure 6b presents the CDF of the number of hops taken by Opera in the ToR-to-ToR optical interconnect. Opera employs k-shortest paths to opportunistically reduce FCTs for short flows, but this results in longer paths, lower throughput, and consequently, reduced link utilization. In contrast, *Vermilion* provides direct single-hop communication between ToR pairs, significantly enhancing link utilization.

4.2 MoE All-to-All Completion Times

Our evaluation in this section is based on packet-level simulations using htsim and Flexflow [22].

Topology: Following prior work [22], we consider a cluster of 128 servers with 1024 GPUs interconnected via an NVSwitch fabric, providing an aggregate intra-server bandwidth of 900 GBps. Each server is also equipped with 8 NICs, each operating at a capacity of c. The NICs are split across two fabrics: 4 NICs are connected to an electrical packet-switched network organized as a fat-tree where each link is 400Gbps, while the remaining 4 NICs are connected to an optical circuit-switched fabric with link capacity c that we vary in the range 800 to 9.6Tbps. The OCS fabric is used to dynamically interconnect GPUs within an expert-parallel group, with a maximum group size of 64 GPUs. This scale is well within the capabilities of commodity optical circuit switches. The OCS incurs a reconfiguration delay of Δ_r that we vary in the range 10ns to 10ms.

Comparisons & configurations: We compare *Vermilion* with oblivious periodic switching using VLB (similar to Sirius [3], RotorNet [4], Shale [8]), Birkhoff decomposition-based schedules, and Mixnet [22]. All systems use the topology described above, and each algorithm modifies the optical switching during MoE dispatch and combine phases involving all-to-all communication. We set $k=3$ for *Vermilion*. Similar to MixNet [22], *Vermilion* and BvN compute switching schedules before the dispatch and combine phases begin in MoE layers.

Workload: We evaluate four MoE models: DeepSeek, Qwen-MoE, Mixtral-8$\times$22B, and Mixtral-8$\times$7B. We use FlexFlow

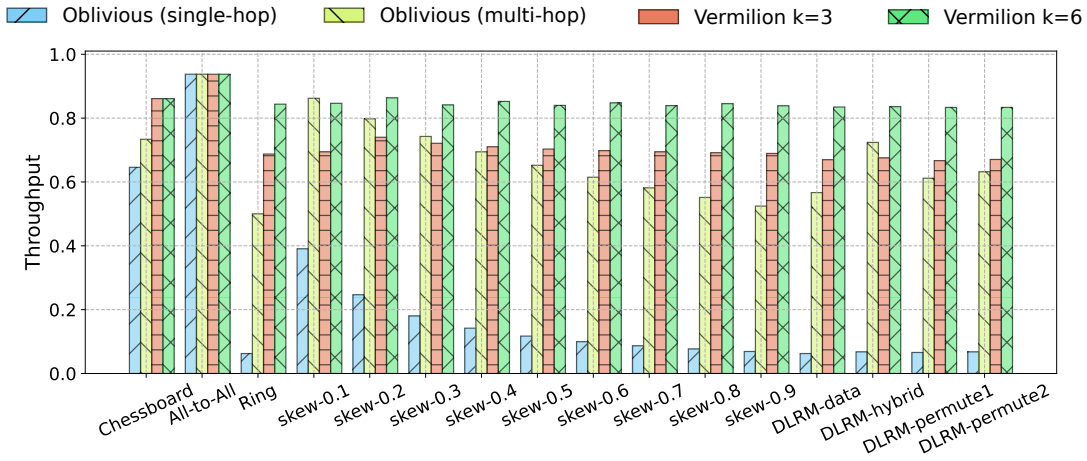
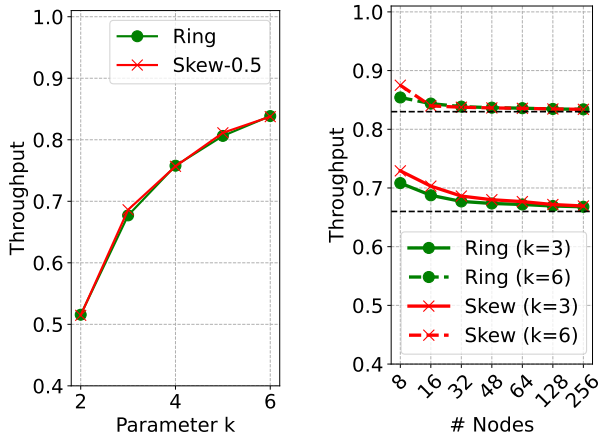


Figure 7: *Vermilion* achieves higher throughput even with single-hop direct communication compared to oblivious networks with multi-hop routing, across real-world demand matrices as well as synthetic demands. The throughput of oblivious periodic networks is severely low when restricted to single-hop routing.



(a) Impact of parameter k (b) Variation with network size

Figure 8: *Vermilion*'s throughput follows its theoretical lower bound with increasing k and converges to the bound as the network size increases.

to generate the MoE execution task graph, which is then used to drive a packet-level network simulation implemented in htsim. We extend the simulator to support *Vermilion*, as well as baseline schemes including oblivious periodic switching with Valiant load balancing and Birkhoff–von Neumann (BvN) schedules. We report the speedup in completion time of the all-to-all communication phases corresponding to the MoE dispatch and combine operations.

As optical bandwidth increases, *Vermilion* achieves higher speedups: Figure 5 shows the average all-to-all completion times for Deepseek MoE. For a fixed reconfiguration delay, *Vermilion* achieves higher speedups as the optical link bandwidth increases. For example, at a 1 μs reconfiguration delay, *Vermilion* achieves a 1.2× speedup over MixNet at 800 Gbps, which increases to 24× at 9.6 Tbps. This trend arises because higher optical bandwidth reduces the number of periods required to complete the all-to-all, allowing *Vermilion* to better leverage its high-throughput capability. Figure 5a shows that *Vermilion* consistently outperforms VLB, with the gap widening as optical bandwidth increases. Notably, the additional queuing at intermediate hops further impacts the completion times for VLB. However, at low reconfiguration delays (between 10 ns and 500 ns), VLB outperforms *Vermilion*, since the reconfiguration interval is small enough to effectively schedule traffic at a per-packet granularity. Finally, Figure 5c shows that *Vermilion* remains close to the optimal throughput achieved by BvN, at low reconfiguration delays. As both the optical bandwidth and the reconfiguration delay increase, *Vermilion* benefits from requiring fewer reconfigurations, translating into higher effective throughput and, in turn, higher speedups even compared to BvN.

***Vermilion* sustains high speedups even under long reconfiguration delays:** As expected, *Vermilion*'s performance degrades as the reconfiguration delay increases. When the reconfiguration delay dominates the transmission time of the demand, a static one-shot configuration becomes preferable. Figure 5 illustrates the tipping point up to which *Vermilion* continues to provide performance benefits over competing approaches. For example as shown in 5b, at 800 Gbps, *Vermilion* outperforms MixNet up to a reconfiguration delay of 100 μs. At 9.6 Tbps, this tipping point increases to 1 ms. Beyond these points, the cost of reconfiguration begins to outweigh the benefits of dynamic scheduling. Nevertheless, *Vermilion* significantly outperforms VLB and BvN at higher reconfiguration delays due to its ability to achieve high throughput while maintaining short schedules, in contrast to BvN which incurs longer schedules.

4.3 Throughput

We now evaluate the throughput capabilities of *Vermilion* in comparison to existing approaches by directly analyzing throughput using a linear programming approach, eliminating

protocol-level interference. We use Gurobi [40] to solve the linear program for throughput maximization.

Topology: We consider a 16 node topology with degree 4 (incoming and outgoing links), interconnected by a layer of optical circuit switches. We set the link capacity to 25Gbps and the reconfiguration delay to $0.5\mu s$ as before. We present our results for a larger network with 48 nodes in Appendix E.

Demand matrices: We evaluate across a variety of demand matrices gathered from a 16 node GPU cluster running distributed training workload of a deep learning recommendation model, under data-parallelism, hybrid parallelism and permutations of the data-parallel workload. We further consider synthetic demand matrices to stress the throughput capabilities of each system; parametrized by a skew parameter that combines a permutation matrix with an all-to-all uniform matrix. For instance, skew-0.1 indicates a 10% skew towards a permutation matrix.

Comparisons: We compare *Vermilion* with an ideal oblivious periodic network that emulates an all-to-all mesh topology, using an ideal routing algorithm that maximizes throughput. We call this system *Oblivious (multi-hop)*. We also compare *Vermilion* with oblivious systems restricted to single-hop. *Vermilion* explicitly uses single-hop routing and we compare $k=3$ and $k=6$.

***Vermilion* consistently achieves high throughput:** From Figure 7, we see that *Vermilion* achieves high throughput across a wide range of communication patterns. Specifically for distributed training workloads, we see that *Vermilion* achieves 6.64% better throughput compared to oblivious periodic networks using an ideal multi-hop routing. Figure 7 shows the clear advantage (and the need) for multi-hop routing in the case of oblivious networks, with significantly lower throughput under single-hop routing. *Vermilion*, on the other hand, achieves high throughput with single-hop routing. Oblivious networks with an ideal routing scheme, however, outperform *Vermilion* when the traffic matrix is close to uniform. This is expected since with $k=3$, *Vermilion*'s lower bound is $\frac{2}{3}$. As the skew increase, oblivious network design drops to a throughput of $\frac{1}{2}$ as discussed in §2.2. In contrast, *Vermilion* maintains a throughput greater than $\frac{2}{3}$ even with skewed demand matrices.

***Vermilion*'s throughput converges to the lower bound:** Figure 8 confirms our theoretical bounds established in §3.2. With increasing k, Figure 8a shows that *Vermilion*'s throughput closely tracks its lower bound of $\frac{k-1}{k}$. Further, even with increasing size of the network, Figure 8b shows that *Vermilion*'s throughput gradually converges to the theoretical lower bound of $\frac{2}{3}$ for $k=3$ and $\frac{5}{6}$ for $k=6$, respectively. This demonstrates the robustness of *Vermilion*'s throughput guarantees across different network sizes and demand matrices.

5 Related Work

Datacenter topologies have been widely studied in the literature both in the context of traditional packet-switched networks [1, 28, 41–47] and emerging reconfigurable optically circuit-switched networks [3–6, 9, 10, 17, 21, 48–59]. In the design of topologies, various metrics of interest have been considered. For instance, uniformly high bandwidth availability [28, 47], expansion [41, 42], fault-tolerance [46], minimizing the link modifications [60] and even the life cycle management of a datacenter [44]. In the context of reconfigurable networks, typically, the goal has been either to minimize the reconfiguration overhead [3, 4] or to minimize the bandwidth tax [5, 6, 9, 32].

Recent works argue for a new measure i.e., "throughput", to understand the maximum load supported by a topology [7, 17, 23, 25, 61]. In fact, the max-flow that relates to the throughput of a topology, can be $O(\log n)$ factor lower than the sparsest cut [25, 62, 63]. Namyar et al. study the throughput upper bound for static datacenter topologies and show a separation between Clos (i.e., fat-trees) and expander-based networks in terms of throughput [23]. In the context of reconfigurable networks, only recently have the throughput bounds of traffic-oblivious networks been established [7, 11, 17].

While throughput of a datacenter topology is interesting from a theory standpoint, a vast majority of the literature focuses on practically achieving the ideal throughput of a topology. For instance, congestion control [39, 64–70], buffer management [71–76], scheduling [77–79], load-balancing [80–82]. In fact, the underlying protocols can turn out to be the key enablers (or limiters) of system performance in the datacenter [67]. Only recently, congestion control [83–85] and routing [86] algorithms tailored for reconfigurable networks have been considered. Interestingly, if *Vermilion* is deployed in an all-optical setting, it does not require any additional congestion control, buffer management and load-balancing mechanisms, since it relies solely on direct communication.

More recently, there has been a growing interest in traffic-aware reconfigurable networks in the context of distributed training and inference [22, 32, 87–92], particularly focusing on resiliency and structured communications driven by AllReduce/AllGather operations that allow pre-scheduling circuits based on the fixed communication pattern. In contrast, *Vermilion* provides theoretical guarantees on its throughput for any traffic matrix and demonstrates its performance across a wide range of workloads and demand matrices.

6 Conclusion

We introduced *Vermilion*, a simple traffic-aware optical interconnect that achieves high throughput using only periodic circuit-switching and direct communication. Through formal analysis, we established throughput bounds for *Vermilion*, marking the first formal separation result that demonstrates traffic-aware reconfigurable networks' superiority over oblivious counterparts in terms of throughput and demand completion times. We believe that *Vermilion* offers a practical solution for datacenter networks with predictable communication patterns. In the future, we plan to explore the temporal dependencies in communication patterns that arise in distributed training workloads and investigate how *Vermilion* can be further optimized for such scenarios.

References

[1] Arjun Singh, Joon Ong, Amit Agarwal, Glen Anderson, Ashby Armistead, Roy Bannon, Seb Boving, Gaurav Desai, Bob Felderman, Paulie Germano, Anand Kanagala, Jeff Provost, Jason Simmons, Eiichi Tanda, Jim Wanderer, Urs Hölzle, Stephen Stuart, and Amin Vahdat. Jupiter rising: A decade of clos topologies and centralized control in google's datacenter network. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, SIGCOMM '15, page 183–197, New York, NY, USA, 2015. Association for Computing Machinery.

[2] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, and Amin Vahdat. Jupiter evolving: transforming google's datacenter network via optical circuit switches and software-defined networking. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, page 66–85, New York, NY, USA, 2022. Association for Computing Machinery.

[3] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwik, Fotini Karinou, Sophie Lange, Kai Shi, Benn Thomsen, and Hugh Williams. Sirius: A flat datacenter network with nanosecond optical switching. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '20, page 782–797, New York, NY, USA, 2020. Association for Computing Machinery.

[4] William M. Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C. Snoeren, and George Porter. Rotornet: A scalable, low-complexity, optical datacenter network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '17, page 267–280, New York, NY, USA, 2017. Association for Computing Machinery.

[5] Monia Ghobadi, Ratul Mahajan, Amar Phanishayee, Nikhil Devanur, Janardhan Kulkarni, Gireeja Ranade, Pierre-Alexandre Blanche, Houman Rastegarfar, Madeleine Glick, and Daniel Kilper. Projector: Agile reconfigurable data center interconnect. In *Proceedings of the 2016 ACM SIGCOMM Conference*, SIGCOMM '16, page 216–229, New York, NY, USA, 2016. Association for Computing Machinery.

[6] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiahu Fainman, George Papen, and Amin Vahdat. Helios: a hybrid electrical/optical switch architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM '10, page 339–350, New York, NY, USA, 2010. Association for Computing Machinery.

[7] Vamsi Addanki, Chen Avin, and Stefan Schmid. Mars: Near-optimal throughput with shallow buffers in reconfigurable datacenter networks. *Proc. ACM Meas. Anal. Comput. Syst.*, 7(1), mar 2023.

[8] Daniel Amir, Nitika Saran, Tegan Wilson, Robert Kleinberg, Vishal Shrivastav, and Hakim Weatherspoon. Shale: A practical, scalable oblivious reconfigurable network. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 449–464, New York, NY, USA, 2024. Association for Computing Machinery.

[9] Johannes Zerwas, Csaba Györgyi, Andreas Blenk, Stefan Schmid, and Chen Avin. Duo: A high-throughput reconfigurable datacenter network using local routing and control. *Proc. ACM Meas. Anal. Comput. Syst.*, 7(1), mar 2023.

[10] William M. Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C. Snoeren, and George Porter. Expanding across time to deliver bandwidth efficiency and low latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 1–18, Santa Clara, CA, February 2020. USENIX Association.

[11] Daniel Amir, Tegan Wilson, Vishal Shrivastav, Hakim Weatherspoon, Robert Kleinberg, and Rachit Agarwal. Optimal oblivious reconfigurable networks. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, page 1339–1352, New York, NY, USA, 2022. Association for Computing Machinery.

[12] George Porter, Richard Strong, Nathan Farrington, Alex Forencich, Pang Chen-Sun, Tajana Rosing, Yeshaiahu Fainman, George Papen, and Amin Vahdat. Integrating microsecond circuit switching into the data center. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM '13, page 447–458, New York, NY, USA, 2013. Association for Computing Machinery.

[13] Zhongkai Yu, Yue Guan, Zihao Yu, Chenyang Zhou, Zhengding Hu, Shuyi Pei, Yangwook Kang, Yufei Ding, and Po-An Tsai. Orders in chaos: Enhancing large-scale moe llm serving with data movement forecasting. *arXiv preprint arXiv:2510.05497*, 2025.

[14] Peizhuang Cong, Aomufei Yuan, Shimao Chen, Yuxuan Tian, Bowen Ye, and Tong Yang. Prediction is all moe needs: Expert load distribution goes from fluctuating to stabilizing. *arXiv preprint arXiv:2404.16914*, 2024.

[15] William M. Mellette, Ilya Agurok, Alex Forencich, Spencer Chang, George Papen, and Joseph E. Ford. A scalable, high-speed optical rotor switch. In *Optical Fiber Communication Conference (OFC) 2024*, page Th1A.5. Optica Publishing Group, 2024.

[16] William M. Mellette, Alex Forencich, Rukshani Athapathu, Alex C. Snoeren, George Papen, and George Porter. Realizing rotornet: Toward practical microsecond scale optical networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 392–414, New York, NY, USA, 2024. Association for Computing Machinery.

[17] Chen Griner, Johannes Zerwas, Andreas Blenk, Manya Ghobadi, Stefan Schmid, and Chen Avin. Cerberus: The power of choices in datacenter topology design - a throughput perspective. *Proc. ACM Meas. Anal. Comput. Syst.*, 5(3), dec 2021.

[18] Garrett Birkhoff. Tres observaciones sobre el algebra lineal. *Univ. Nac. Tucuman, Ser. A*, 5:147–154, 1946.

[19] He Liu, Matthew K. Mukerjee, Conglong Li, Nicolas Feltman, George Papen, Stefan Savage, Srinivasan Seshan, Geoffrey M. Voelker, David G. Andersen, Michael Kaminsky, George Porter, and Alex C. Snoeren. Scheduling techniques for hybrid circuit/packet networks. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies*, CoNEXT '15, New York, NY, USA, 2015. Association for Computing Machinery.

[20] Michael Bacharach. Matrix rounding problems. *Management Science*, 12(9):732–742, 1966.

[21] Shaileshh Bojja Venkatakrishnan, Mohammad Alizadeh, and Pramod Viswanath. Costly circuits, submodular schedules and approximate carathéodory theorems. In *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, SIGMETRICS '16, page 75–88, New York, NY, USA, 2016. Association for Computing Machinery.

[22] Xudong Liao, Yijun Sun, Han Tian, Xinchen Wan, Yilun Jin, Zilong Wang, Zhenghang Ren, Xinyang Huang, Wenxue Li, Kin Fai Tse, Zhizhen Zhong, Guyue Liu, Ying Zhang, Xiaofeng Ye, Yiming Zhang, and Kai Chen. Mixnet: A runtime reconfigurable optical-electrical fabric for distributed mixture-of-experts training. In *Proceedings of the ACM SIGCOMM 2025 Conference*, SIGCOMM '25, page 554–574, New York, NY, USA, 2025. Association for Computing Machinery.

[23] Pooria Namyar, Sucha Supittayapornpong, Mingyang Zhang, Minlan Yu, and Ramesh Govindan. A throughput-centric view of the performance of datacenter topologies. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference*, SIGCOMM '21, page 349–369, New York, NY, USA, 2021. Association for Computing Machinery.

[24] N. G. Duffield, Pawan Goyal, Albert Greenberg, Partho Mishra, K. K. Ramakrishnan, and Jacobus E. van der Merive. A flexible model for resource management in virtual private networks. In *Proceedings of the Conference on Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '99, page 95–108, New York, NY, USA, 1999. Association for Computing Machinery.

[25] Sangeetha Abdu Jyothi, Ankit Singla, P. Brighten Godfrey, and Alexandra Kolla. Measuring and understanding throughput of network topologies. In *SC '16: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 761–772, 2016.

[26] Farhad Shahrokhi and D. W. Matula. The maximum concurrent flow problem. *J. ACM*, 37(2):318–334, apr 1990.

[27] Janardhan Kulkarni, Stefan Schmid, and Paweł Schmidt. Scheduling opportunistic links in two-tiered reconfigurable datacenters. In *Proceedings of the 33rd ACM Symposium on Parallelism in Algorithms and Architectures*, SPAA '21, page 318–327, New York, NY, USA, 2021. Association for Computing Machinery.

[28] Mohammad Al-Fares, Alexander Loukissas, and Amin Vahdat. A scalable, commodity data center network architecture. In *Proceedings of the ACM SIGCOMM 2008 Conference on Data Communication*, SIGCOMM '08, page 63–74, New York, NY, USA, 2008. Association for Computing Machinery.

[29] Janardhan Kulkarni, Euiwoong Lee, and Mohit Singh. Minimum birkhoff-von neumann decomposition. In Friedrich Eisenbrand and Jochen Koenemann, editors, *Integer Programming and Combinatorial Optimization*, pages 343–354, Cham, 2017. Springer International Publishing.

[30] William Maxwell Mellette, Glenn M. Schuster, George Porter, George Papen, and Joseph E. Ford. A scalable, partially configurable optical switch for data center networks. *Journal of Lightwave Technology*, 35(2):136–144, 2017.

[31] Polatis. POLATIS® 7000 Optical Circuit Switch. https://www.hubersuhner.com/en/shop/product/other-systems/optical-switches/rack-mount-circuit-switches/85223159/polatis-7000-optical-circuit-switch.

[32] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. TopoOpt: Co-optimizing network topology and parallelization strategy for distributed training jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 739–767, Boston, MA, April 2023. USENIX Association.

[33] Wenxue Li, Xiangzhou Liu, Yuxuan Li, Yilun Jin, Han Tian, Zhizhen Zhong, Guyue Liu, Ying Zhang, and Kai Chen. Understanding communication characteristics of distributed training. In *Proceedings of the 8th Asia-Pacific Workshop on Networking*, APNet '24, page 1–8, New York, NY, USA, 2024. Association for Computing Machinery.

[34] Weiyang Wang, Manya Ghobadi, Kayvon Shakeri, Ying Zhang, and Naader Hasani. Rail-only: A low-cost high-performance network for training llms with trillion parameters. In *2024 IEEE Symposium on High-Performance Interconnects (HOTI)*, pages 1–10, Los Alamitos, CA, USA, aug 2024. IEEE Computer Society.

[35] opera sim. https://github.com/TritonNetworking/opera-sim.

[36] Cong Liang, Xiangli Song, Jing Cheng, Mowei Wang, Yashe Liu, Zhenhua Liu, Shizhen Zhao, and Yong Cui. Negotiator: Towards a simple yet effective on-demand reconfigurable datacenter network. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 415–432, New York, NY, USA, 2024. Association for Computing Machinery.

[37] Leslie G. Valiant. A scheme for fast parallel communication. *SIAM journal on computing*, 11(2):350–361, 1982.

[38] Mark Handley, Costin Raiciu, Alexandru Agache, Andrei Voinescu, Andrew W. Moore, Gianni Antichi, and Marcin Wójcik. Re-architecting datacenter networks and stacks for low latency and high performance. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '17, page 29–42, New York, NY, USA, 2017. Association for Computing Machinery.

[39] Mohammad Alizadeh, Albert Greenberg, David A. Maltz, Jitendra Padhye, Parveen Patel, Balaji Prabhakar, Sudipta Sengupta, and Murari Sridharan. Data center tcp (dctcp). In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM '10, page 63–74, New York, NY, USA, 2010. Association for Computing Machinery.

[40] Gurobi Optimization, LLC. Gurobi Optimizer Reference Manual, 2023.

[41] Asaf Valadarsky, Gal Shahaf, Michael Dinitz, and Michael Schapira. Xpander: Towards optimal-performance datacenters. In *Proceedings of the 12th International on Conference on Emerging Networking EXperiments and Technologies*, CoNEXT '16, page 205–219, New York, NY, USA, 2016. Association for Computing Machinery.

[42] Ankit Singla, Chi-Yao Hong, Lucian Popa, and P. Brighten Godfrey. Jellyfish: Networking data centers randomly. In *9th USENIX Symposium on Networked Systems Design and Implementation (NSDI 12)*, pages 225–238, San Jose, CA, April 2012. USENIX Association.

[43] Albert Greenberg, James R. Hamilton, Navendu Jain, Srikanth Kandula, Changhoon Kim, Parantap Lahiri, David A. Maltz, Parveen Patel, and Sudipta Sengupta. Vl2: a scalable and flexible data center network. In *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*, SIGCOMM '09, page 51–62, New York, NY, USA, 2009. Association for Computing Machinery.

[44] Mingyang Zhang, Radhika Niranjan Mysore, Sucha Supittayapornpong, and Ramesh Govindan. Understanding lifecycle management complexity of datacenter topologies. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*, pages 235–254, Boston, MA, February 2019. USENIX Association.

[45] Maciej Besta and Torsten Hoefler. Slim fly: A cost effective low-diameter network topology. In *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 348–359, 2014.

[46] Vincent Liu, Daniel Halperin, Arvind Krishnamurthy, and Thomas Anderson. F10: A Fault-Tolerant engineered network. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*, pages 399–412, Lombard, IL, April 2013. USENIX Association.

[47] Chuanxiong Guo, Guohan Lu, Dan Li, Haitao Wu, Xuan Zhang, Yunfeng Shi, Chen Tian, Yongguang Zhang, and Songwu Lu. Bcube: a high performance, server-centric network architecture for modular data centers. In *Proceedings of the ACM SIGCOMM 2009*

Conference on Data Communication, SIGCOMM '09, page 63–74, New York, NY, USA, 2009. Association for Computing Machinery.

[48] Xia Zhou, Zengbin Zhang, Yibo Zhu, Yubo Li, Saipriya Kumar, Amin Vahdat, Ben Y. Zhao, and Haitao Zheng. Mirror mirror on the ceiling: flexible wireless links for data centers. *SIGCOMM Comput. Commun. Rev.*, 42(4):443–454, aug 2012.

[49] Srikanth Kandula, Jitendra Padhye, and Paramvir Bahl. Flyways to de-congest data center networks. In *HotNets*. ACM SIGCOMM, 2009.

[50] Navid Hamedazimi, Zafar Qazi, Himanshu Gupta, Vyas Sekar, Samir R. Das, Jon P. Longtin, Himanshu Shah, and Ashish Tanwer. Firefly: a reconfigurable wireless data center fabric using free-space optics. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, page 319–330, New York, NY, USA, 2014. Association for Computing Machinery.

[51] Li Chen, Kai Chen, Zhonghua Zhu, Minlan Yu, George Porter, Chunming Qiao, and Shan Zhong. Enabling Wide-Spread communications on optical fabric with MegaSwitch. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*, pages 577–593, Boston, MA, March 2017. USENIX Association.

[52] Yunpeng James Liu, Peter Xiang Gao, Bernard Wong, and Srinivasan Keshav. Quartz: a new design element for low-latency dcns. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, page 283–294, New York, NY, USA, 2014. Association for Computing Machinery.

[53] Kai Chen, Ankit Singla, Atul Singh, Kishore Ramachandran, Lei Xu, Yueping Zhang, Xitao Wen, and Yan Chen. Osa: An optical switching architecture for data center networks with unprecedented flexibility. *IEEE/ACM Transactions on Networking*, 22(2):498–511, 2014.

[54] Guohui Wang, David G. Andersen, Michael Kaminsky, Konstantina Papagiannaki, T.S. Eugene Ng, Michael Kozuch, and Michael Ryan. c-through: part-time optics in data centers. In *Proceedings of the ACM SIGCOMM 2010 Conference*, SIGCOMM '10, page 327–338, New York, NY, USA, 2010. Association for Computing Machinery.

[55] Stefan Schmid, Chen Avin, Christian Scheideler, Michael Borokhovich, Bernhard Haeupler, and Zvi Lotker. Splaynet: Towards locally self-adjusting networks. *IEEE/ACM Transactions on Networking*, 24(3):1421–1433, 2016.

[56] Ankit Singla, Atul Singh, Kishore Ramachandran, Lei Xu, and Yueping Zhang. Proteus: a topology malleable data center network. In *Proceedings of the 9th ACM SIGCOMM Workshop on Hot Topics in Networks*, Hotnets-IX, New York, NY, USA, 2010. Association for Computing Machinery.

[57] Weitao Wang, Dingming Wu, Sushovan Das, Afsaneh Rahbar, Ang Chen, and T. S. Eugene Ng. RDC: Energy-Efficient data center network congestion relief with topological reconfigurability at the edge. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 1267–1288, Renton, WA, April 2022. USENIX Association.

[58] Klaus-Tycho Foerster and Stefan Schmid. Survey of reconfigurable data center networks: Enablers, algorithms, complexity. *SIGACT News*, 50(2):62–79, jul 2019.

[59] Kevin Liang, Litao Qiao, Isaac Keslassy, and Bill Lin. Scheduling parallel optical switches in datacenters. *IEEE Networking Letters*, 8:149–153, 2026.

[60] Zihan Zhu, Xinchi Han, Dongchao Wu, Zhanbang Zhang, Jian Yang, Shizhen Zhao, and Xinbing Wang. Fastrechain: A novel bidirectional model-based algorithm for topology engineering of ocs-based clusters, 2026.

[61] Ankit Singla, P. Brighten Godfrey, and Alexandra Kolla. High throughput data center topology design. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, pages 29–41, Seattle, WA, April 2014. USENIX Association.

[62] Tom Leighton and Satish Rao. Multicommodity max-flow min-cut theorems and their use in designing approximation algorithms. *J. ACM*, 46(6):787–832, nov 1999.

[63] T. Leighton and S. Rao. An approximate max-flow min-cut theorem for uniform multicommodity flow problems with applications to approximation algorithms. In *Proceedings of the 29th Annual Symposium on Foundations of Computer Science*, SFCS '88, page 422–431, USA, 1988. IEEE Computer Society.

[64] Radhika Mittal, Vinh The Lam, Nandita Dukkipati, Emily Blem, Hassan Wassel, Monia Ghobadi, Amin Vahdat, Yaogong Wang, David Wetherall, and David Zats. Timely: Rtt-based congestion control for the datacenter. In *Proceedings of the 2015 ACM Conference on Special Interest Group on Data Communication*, SIGCOMM '15, page 537–550, New York, NY, USA, 2015. Association for Computing Machinery.

[65] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming

Zhang, Frank Kelly, Mohammad Alizadeh, and Minlan Yu. Hpcc: High precision congestion control. In *Proceedings of the ACM Special Interest Group on Data Communication*, SIGCOMM '19, page 44–58, New York, NY, USA, 2019. Association for Computing Machinery.

[66] Vamsi Addanki, Oliver Michel, and Stefan Schmid. PowerTCP: Pushing the performance limits of datacenter networks. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 51–70, Renton, WA, April 2022. USENIX Association.

[67] Gautam Kumar, Nandita Dukkipati, Keon Jang, Hassan M. G. Wassel, Xian Wu, Behnam Montazeri, Yaogong Wang, Kevin Springborn, Christopher Alfeld, Michael Ryan, David Wetherall, and Amin Vahdat. Swift: Delay is simple and effective for congestion control in the datacenter. In *Proceedings of the Annual Conference of the ACM Special Interest Group on Data Communication on the Applications, Technologies, Architectures, and Protocols for Computer Communication*, SIGCOMM '20, page 514–528, New York, NY, USA, 2020. Association for Computing Machinery.

[68] Prateesh Goyal, Preey Shah, Kevin Zhao, Georgios Nikolaidis, Mohammad Alizadeh, and Thomas E. Anderson. Backpressure flow control. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*, pages 779–805, Renton, WA, April 2022. USENIX Association.

[69] Ahmed Saeed, Varun Gupta, Prateesh Goyal, Milad Sharif, Rong Pan, Mostafa Ammar, Ellen Zegura, Keon Jang, Mohammad Alizadeh, Abdul Kabbani, and Amin Vahdat. Annulus: A dual congestion control loop for datacenter and wan traffic aggregates. SIGCOMM '20, page 735–749, New York, NY, USA, 2020. Association for Computing Machinery.

[70] Christo Wilson, Hitesh Ballani, Thomas Karagiannis, and Ant Rowtron. Better never than late: meeting deadlines in datacenter networks. In *Proceedings of the ACM SIGCOMM 2011 Conference*, SIGCOMM '11, page 50–61, New York, NY, USA, 2011. Association for Computing Machinery.

[71] Vamsi Addanki, Maria Apostolaki, Manya Ghobadi, Stefan Schmid, and Laurent Vanbever. Abm: Active buffer management in datacenters. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, page 36–52, New York, NY, USA, 2022. Association for Computing Machinery.

[72] Maria Apostolaki, Laurent Vanbever, and Manya Ghobadi. Fab: Toward flow-aware buffer sharing on programmable switches. In *Proceedings of the 2019 Workshop on Buffer Sizing*, BS '19, New York, NY, USA, 2020. Association for Computing Machinery.

[73] Sijiang Huang, Mowei Wang, and Yong Cui. Traffic-aware buffer management in shared memory switches. *IEEE/ACM Transactions on Networking*, 30(6):2559–2573, 2022.

[74] Vamsi Addanki, Wei Bai, Stefan Schmid, and Maria Apostolaki. Reverie: Low pass Filter-Based switch buffer sharing for datacenters with RDMA and TCP traffic. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 651–668, Santa Clara, CA, April 2024. USENIX Association.

[75] Hamidreza Almasi, Rohan Vardekar, and Balajee Vamanan. Protean: Adaptive management of shared-memory in datacenter switches. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pages 1–10, 2023.

[76] Vamsi Addanki, Maciej Pacut, and Stefan Schmid. Credence: Augmenting datacenter switch buffer sharing with ML predictions. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 613–634, Santa Clara, CA, April 2024. USENIX Association.

[77] Mohammad Alizadeh, Shuang Yang, Milad Sharif, Sachin Katti, Nick McKeown, Balaji Prabhakar, and Scott Shenker. Pfabric: Minimal near-optimal datacenter transport. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM*, SIGCOMM '13, page 435–446, New York, NY, USA, 2013. Association for Computing Machinery.

[78] Mohammad Al-Fares, Sivasankar Radhakrishnan, Barath Raghavan, Nelson Huang, and Amin Vahdat. Hedera: Dynamic flow scheduling for data center networks. In *7th USENIX Symposium on Networked Systems Design and Implementation (NSDI 10)*, San Jose, CA, April 2010. USENIX Association.

[79] Sudarsanan Rajasekaran, Manya Ghobadi, and Aditya Akella. Cassini: Network-aware job scheduling in machine learning clusters. In *21th USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, Santa Clara, CA, 2024. USENIX Association.

[80] Mohammad Alizadeh, Tom Edsall, Sarang Dharmapurikar, Ramanan Vaidyanathan, Kevin Chu, Andy Fingerhut, Vinh The Lam, Francis Matus, Rong Pan, Navindra Yadav, and George Varghese. Conga: Distributed congestion-aware load balancing for datacenters. In *Proceedings of the 2014 ACM Conference on SIGCOMM*, SIGCOMM '14, page 503–514, New York, NY, USA, 2014. Association for Computing Machinery.

[81] Naga Katta, Mukesh Hira, Changhoon Kim, Anirudh Sivaraman, and Jennifer Rexford. Hula: Scalable load balancing using programmable data planes. In *Proceedings of the Symposium on SDN Research*, SOSR '16, New York, NY, USA, 2016. Association for Computing Machinery.

[82] Soudeh Ghorbani, Zibin Yang, P. Brighten Godfrey, Yashar Ganjali, and Amin Firoozshahian. Drill: Micro load balancing for low-latency data center networks. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*, SIGCOMM '17, page 225–238, New York, NY, USA, 2017. Association for Computing Machinery.

[83] Daniel Amir, Tegan Wilson, Vishal Shrivastav, Hakim Weatherspoon, and Robert Kleinberg. Poster: Scalability and congestion control in oblivious reconfigurable networks. In *Proceedings of the ACM SIGCOMM 2023 Conference*, ACM SIGCOMM '23, page 1138–1140, New York, NY, USA, 2023. Association for Computing Machinery.

[84] Shawn Shuoshuo Chen, Weiyang Wang, Christopher Canel, Srinivasan Seshan, Alex C. Snoeren, and Peter Steenkiste. Time-division tcp for reconfigurable data center networks. In *Proceedings of the ACM SIGCOMM 2022 Conference*, SIGCOMM '22, page 19–35, New York, NY, USA, 2022. Association for Computing Machinery.

[85] Matthew K. Mukerjee, Christopher Canel, Weiyang Wang, Daehyeok Kim, Srinivasan Seshan, and Alex C. Snoeren. Adapting TCP for reconfigurable datacenter networks. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*, pages 651–666, Santa Clara, CA, February 2020. USENIX Association.

[86] Jialong Li, Haotian Gong, Federico De Marchi, Aoyu Gong, Yiming Lei, Wei Bai, and Yiting Xia. Uniform-cost multi-path routing for reconfigurable data center networks. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 433–448, New York, NY, USA, 2024. Association for Computing Machinery.

[87] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. Tpu v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, ISCA '23, New York, NY, USA, 2023. Association for Computing Machinery.

[88] Yazhou Zu, Alireza Ghaffarkhah, Hoang-Vu Dang, Brian Towles, Steven Hand, Safeen Huda, Adekunle Bello, Alexander Kolbasov, Arash Rezaei, Dayou Du, Steve Lacy, Hang Wang, Aaron Wisner, Chris Lewis, and Henri Bahini. Resiliency at scale: Managing Google's TPUv4 machine learning supercomputer. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*, pages 761–774, Santa Clara, CA, April 2024. USENIX Association.

[89] Vamsi Addanki. When light bends to the collective will: A theory and vision for adaptive photonic scale-up domains. In *Proceedings of the 24th ACM Workshop on Hot Topics in Networks*, HotNets '25, page 326–334, New York, NY, USA, 2025. Association for Computing Machinery.

[90] Abhishek Vijaya Kumar, Arjun Devraj, and Rachee Singh. Pccl: Photonic circuit-switched collective communication for distributed ml, 2025.

[91] Abhishek Vijaya Kumar, Eric Ding, Arjun Devraj, Darius Bunandar, and Rachee Singh. Lumion: Fast fault recovery for ml jobs using programmable optical fabrics. *arXiv preprint arXiv:2505.23105*, 2025.

[92] Liangyu Zhao, Siddharth Pal, Tapan Chugh, Weiyang Wang, Jason Fantl, Prithwish Basu, Joud Khoury, and Arvind Krishnamurthy. Efficient Direct-Connect topologies for collective communications. In *22nd USENIX Symposium on Networked Systems Design and Implementation (NSDI 25)*, pages 705–737, Philadelphia, PA, April 2025. USENIX Association.

[93] Wei Bai, Shanim Sainul Abdeen, Ankit Agrawal, Krishan Kumar Attre, Paramvir Bahl, Ameya Bhagat, Gowri Bhaskara, Tanya Brokhman, Lei Cao, Ahmad Cheema, Rebecca Chow, Jeff Cohen, Mahmoud Elhaddad, Vivek Ette, Igal Figlin, Daniel Firestone, Mathew George, Ilya German, Lakhmeet Ghai, Eric Green, Albert Greenberg, Manish Gupta, Randy Haagens, Matthew Hendel, Ridwan Howlader, Neetha John, Julia Johnstone, Tom Jolly, Greg Kramer, David Kruse, Ankit Kumar, Erica Lan, Ivan Lee, Avi Levy, Marina Lipshteyn, Xin Liu, Chen Liu, Guohan Lu, Yuemin Lu, Xiakun Lu, Vadim Makhervaks, Ulad Malashanka, David A. Maltz, Ilias Marinos, Rohan Mehta, Sharda Murthi, Anup Namdhari, Aaron Ogus, Jitendra Padhye, Madhav Pandya, Douglas Phillips, Adrian Power, Suraj Puri, Shachar Raindel, Jordan Rhee, Anthony Russo, Maneesh Sah, Ali Sheriff, Chris Sparacino, Ashutosh Srivastava, Weixiang Sun, Nick Swanson, Fuhou Tian, Lukasz Tomczyk, Vamsi Vadlamuri, Alec Wolman, Ying Xie, Joyce Yom, Lihua Yuan, Yanzhao Zhang, and Brian Zill. Empowering azure storage with RDMA. In *20th USENIX Symposium on Networked Systems*

Design and Implementation (NSDI 23), pages 49–67, Boston, MA, April 2023. USENIX Association.

[94] V. Yokar, A. Mehrpooya, Y. Teng, S. Shen, Z. Wu, K. Bardhi, S. Yan, and D. Simeonidou. Fast link recovery via ptp-synchronized nanosecond optical switching, 2024.

[95] Yiming Lei, Jialong Li, Zhengqing Liu, Raj Joshi, and Yiting Xia. Nanosecond time synchronization for optical data center networks. In *Proceedings of the ACM SIGCOMM 2025 Posters and Demos*, ACM SIGCOMM Posters and Demos '25, page 40–42, New York, NY, USA, 2025. Association for Computing Machinery.

[96] Alex Forencich, Alex C. Snoeren, George Porter, and George Papen. Corundum: An open-source 100-gbps nic. In *2020 IEEE 28th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*, pages 38–46, 2020.

[97] Kari A. Clark, Daniel Cletheroe, Thomas Gerard, István Haller, Krzysztof Jozwik, Kai Shi, Benn Charles Thomsen, Hugh Williams, Georgios S. Zervas, Hitesh Ballani, Polina Bayvel, Paolo Costa, and Zhixin Liu. Synchronous subnanosecond clock and data recovery for optically switched data centres using clock phase caching. *Nature Electronics*, 3:426–433, 2020.

[98] Thomas Gerard, Kari Clark, Adam Funnell, Kai Shi, Benn Thomsen, Philip Watts, Krzysztof Jozwik, Istvan Haller, Hugh Williams, Paolo Costa, and Hitesh Ballani. Fast and uniform optically-switched data centre networks enabled by amplitude caching. In *2021 Optical Fiber Communications Conference and Exhibition (OFC)*, pages 1–3, 2021.

[99] https://docs.nvidia.com/networking/display/nvidia5ttechnologyusermanualv10.

[100] Yuliang Li, Gautam Kumar, Hema Hariharan, Hassan Wassel, Peter H. Hochschild, Dave Platt, Simon Sabato, Minlan Yu, Nandita Dukkipati, Prashant Chandra, and Amin Vahdat. Sundial: Fault-tolerant clock synchronization for datacenters. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 1171–1186, 2020.

[101] Ken-Ichi Sato. Optical switching will innovate intra data center networks. In *2023 Optical Fiber Communications Conference and Exhibition (OFC)*, pages 1–40, 2023.

[102] Benjamin G. Lee and Nicolas Dupuis. Silicon photonic switch fabrics: Technology and architecture. *Journal of Lightwave Technology*, 37(1):6–20, 2019.

[103] Arslan Sajid Raja, Sophie Lange, Maxim Karpov, Kai Shi, Xin Fu, Raphael Behrendt, Daniel Cletheroe, Anton Lukashchuk, Istvan Haller, Fotini Karinou, Benn Thomsen, Krzysztof Jozwik, Junqiu Liu, Paolo Costa, Tobias Jan Kippenberg, and Hitesh Ballani. Ultrafast optical circuit switching for data centers using integrated soliton microcombs. *Nature Communications*, 12(1), October 2021.

[104] Amirmahdi Honardoost, Johannes Henriksson, Kyungmok Kwon, Jianheng Luo, and Ming C. Wu. Low-loss wafer-bonded silicon photonic mems switches. In *2022 Optical Fiber Communications Conference and Exhibition (OFC)*, pages 1–3, 2022.

[105] Xuting Liu, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. Rethinking machine learning collective communication as a multi-commodity flow problem. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 16–37, New York, NY, USA, 2024. Association for Computing Machinery.

A Frequently Asked Questions

Q1: *How is traffic matrix estimated and at what timescale?*

A1: Demand matrix can be predicted in advance in certain workloads e.g., distributed training [22]. However, traffic matrix estimation is a common challenge in storage and search workloads. In our current design, each node maintains its own traffic estimation towards every other node in the network. During the round-robin phase of *Vermilion*'s circuit-switching, nodes participate in an AllGather operation pipelined with other data transfers during this phase. The AllGather operation runs as follows:

- Each node maintains a local array of traffic estimates (in bits) towards every other node in the network i.e., a single row of the global traffic matrix.
- Each node retrieves its local array of outgoing traffic estimates corresponding to all the destination nodes in the network and transforms the array in two steps. First, it performs normalization by multiplying each entry in the array by $\frac{k-1}{k} \cdot \frac{1}{c \cdot \Delta}$, where c is the physical link capacity, k is the number of phases in a period (a parameter to *Vermilion*) and Δ is the timeslot duration[2]. Second, it rounds down the normalized entries. Importantly, the left over non-zero entries at each node are bounded by the total number of nodes n in the network. Further, each entry's value and the sum of all entries in the array are

[2] $c \cdot \Delta$ corresponds to the total amount of bits that each node can transmit in a single timeslot, which is known in advance and remains constant i.e., based on the physical link capacity and the timeslot duration of the network.

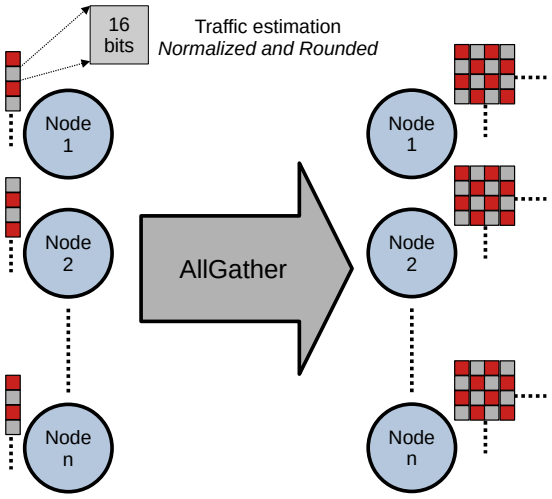


Figure 9: Nodes exchange their local traffic estimates during the round-robin phase of *Vermilion*'s schedule by performing an AllGather operation. At the end of the round-robin phase, each node has an overview of the global traffic matrix and performs the schedule computation in a distributed manner.

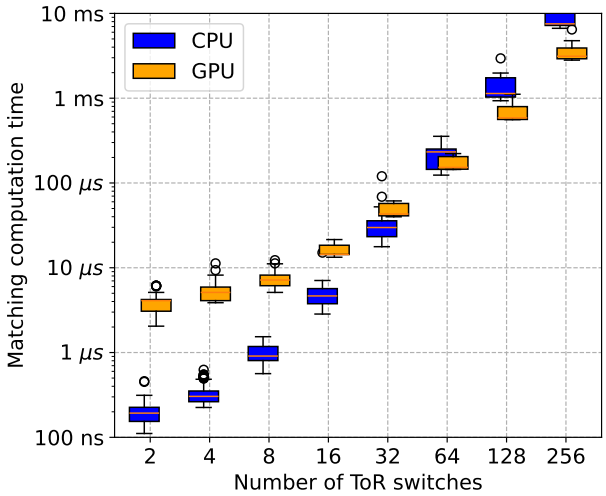


Figure 10: The absolute amount of time required for computing the circuit-switching schedule of *Vermilion* is within microsecond scale for moderate network sizes e.g., with 32 nodes (ToRs) and 32 servers per ToR, *Vermilion* incurs 59μs of computation time on average for a network size of 1024 servers.

bounded by $k \cdot n$. We allocate 16 bits to represent each entry, supporting up to 65536 values corresponding to up to $n = 21845$ number of ToRs in the network.

- During the round-robin phase, each node sends its array of traffic estimations to its direct neighbor in every timeslot — essentially an AllGather operation.
- At the end of the round-robin phase, every node has an overview of the global traffic matrix (normalized and rounded) as shown in Figure 9.

The amount of data transmitted by each node during a single timeslot in the round-robin phase is limited by the number of nodes n. For example, even with a large-scale network of $n = 256$ ToRs, the total data transmitted by a node in a single timeslot amounts to 256×16 bits. With a link capacity of 800 Gbps, this transmission requires only ≈ 41 nanoseconds. Encouragingly, port bandwidth tends to increase over time, further reducing the time required to transmit traffic estimates. This trend suggests that even a conservative estimate of 41 nanoseconds for the minimum timeslot duration for circuit-switching is sufficient for *Vermilion* to efficiently perform the AllGather operation and construct the traffic matrix at each node.

Q2: *How do nodes estimate their local traffic array?*

A2: Each source ToR switch maintains per-destination (ToR) virtual output queues (VOQs). Each entry in the local traffic estimate array at each ToR represents the total number of bits received by the corresponding VOQ. These counters are reset at the end of each round-robin phase, once the transmission of traffic estimates is complete.

Looking ahead, we aim to explore the feasibility of servers reporting local traffic estimates directly to their upstream ToR switches. Server NICs, particularly in RDMA environments, provide more accurate traffic estimates. Specifically, NICs can scan RDMA queue pairs to calculate the total number of bytes specified by the outstanding WQEs across all active queue pairs. This practice, widely used in production for congestion control, telemetry, and buffer management [93], offers an intriguing advantage. Notably, WQEs do not represent bytes waiting for immediate transmission but rather the *outstanding* bytes the NIC *intends* to transmit in the future. This forward-looking information provides a more accurate traffic estimate compared to relying solely on VOQs at the ToR switch. We leave a deeper investigation of this approach for future work.

Q3: *What is the absolute time required to compute Vermilion's schedule?*

A3: Figure 10 shows the time taken to compute *Vermilion*'s schedule for a range of network sizes between 2 to 256 ToR switches. We wrote a CUDA program that takes a doubly stochastic matrix as input and computes the matching decomposition. We tested the computation time on an NVIDIA RTX A2000 GPU. More advanced GPUs with higher compute capabilities can further reduce the computation time.

We observe that the time taken to compute the matching decomposition is within microsecond scale for moderate network sizes. In the following, we consider 32 servers per ToR switch. For instance, Broadcom Tomahawk 5 supports 64 ports of 800Gbps each. With a similar ToR switch radix, 32 servers per ToR is a reasonable assumption. Figure 10 shows that for 32 ToRs (i.e, 1024 servers) in the network, *Vermilion* only incurs 59μs on average for computing the switching schedule. As the network size increases, the time required for computing the

switching schedule increases significantly beyond acceptable limits. In this regard, we currently target up to 32 ToR network size if the optical switches are deployed at the spine layer. For larger network sizes, *Vermilion* may be best suited for circuit-switching at the core, i.e., between spine (or aggregation) and the core switches, similar to Google's Jupiter [2]. Better algorithms for matching decomposition, as well as hardware acceleration, could further reduce the computation time and extend the network size that *Vermilion* can support.

Q4: *How frequently is the new schedule computed and how does it impact the freshness of traffic estimates?*

A4: *Vermilion* performs the traffic matrix AllGather operation in *every* period during the round-robin phase as described in (A1). However, the computations for the matching decomposition are performed asynchronously.

- **Frequency of computations:** Given that the computation happens asynchronously, the frequency of the computations is dictated by the computation time. As shown in Figure 10, the computation time is within microsecond scale for moderate network sizes e.g., 59μs for a network size of 32 ToR switches. This allows for frequent updates to the schedule, ensuring that the network adapts to changing traffic patterns even at microsecond timescales. *Vermilion* updates its schedule only after the computation results are ready and does not block the circuits and communication. We enforce that such an update only happens at the end of a round-robin phase, allowing new computations to account for the most recent traffic estimates gathered by the previous round-robin phase.
- **Time synchronization and Link recovery:** Dynamic optical circuit-switched networks critically rely on time synchronization, including a vast majority of the designs in the literature [3, 4, 8, 10, 16]. In comparison to prior works, *Vermilion* additionally requires that any updates to the circuit-switching schedule happen in a precisely synchronized manner. This synchronization is crucial to prevent inconsistencies in the schedule and to avoid undesirable optical collisions. Accurate time synchronization has been demonstrated experimentally in the literature [3, 94, 95]. In addition, transmission of packets to the optical network needs to be time-synchronous, to ensure data is transmitted when the appropriate optical channel is setup [96]. Link bring-up (clock/data recovery CDR) overheads after each reconfiguration of the network needs to be minimized and relevant approaches have been reported in the literature [3, 16, 94, 97, 98]. Notably, recent developments of commercial solutions [99, 100] can be leveraged to address these challenges. We leave it for future work to explore the synchronization and CDR mechanisms in more detail.
- **Freshness of traffic estimates:** The AllGather operation collects the traffic estimates during every robin-robin phase but computations are performed less frequently in an asynchronous manner. We apply exponential weighted moving averages for each entry in the local traffic estimates at each node. This preserves the freshness of traffic estimates, as well as, takes into account the historical estimates received between two computations.

Q5: *Any variable-duration schedule obtained from BvN decomposition can be converted to a periodic fixed-duration schedule by time quantization. Why is Vermilion's periodic schedule different?*

A5: It is certainly true that any BvN schedule can be converted to a periodic fixed-duration schedule by time quantization. However, such an approach has two critical issues in terms of performance under realistic reconfiguration delays, in contrast to the ideal performance as indicated by Theorem 1:

- **Schedule length:** The length of the schedule obtained from BvN decomposition can be up to n^2-2 matchings, in comparison to just $3 \cdot n$ matchings (for $k = 3$) in *Vermilion*. This results in a significantly longer schedule length for large networks. For instance with $n = 32$, a BvN schedule could produce up to 1024 matchings, whereas, *Vermilion* consistently uses only a period length of 96 which scales down proportionately with the number of physical links (degree) at each ToR switch.
- **Throughput:** More critically, time quantization of a BvN schedule can result in significantly lower throughput due to the reduced circuit duty cycle. For instance, consider 4 matchings produced by BvN for a traffic matrix $\mathcal{M}$, i.e., $\mathcal{M} = \lambda_1 \cdot P_1 + \lambda_2 \cdot P_2 + \lambda_3 \cdot P_3 + \lambda_4 \cdot P_4$. To achieve full throughput in an ideal scenario with zero reconfiguration delay (see Theorem 1), it is necessary to spend $t_1 = \frac{\lambda_1}{\sum_i \lambda}$ fraction of time executing the matching P_1, $t_2 = \frac{\lambda_2}{\sum_i \lambda}$ fraction of time executing the matching P_2, and so on.

  For clarity, we first define *fixed-duration* and *variable-duration* circuit-switching.

  Definition 3 (Fixed-duration circuit-switching)**.** *Any circuit-switching schedule is a sequence of matchings $P = \langle P_1, P_2, \ldots \rangle$. A fixed-duration circuit-switching schedule specifies that every matching is executed for a fixed duration Δ for all matchings $P_i \in P$.*

  For example, in a 4-node network with nodes labeled A, B, C, and D, suppose we desire a circuit $A \to C$ for a duration of 2 timeslots and a circuit $B \to D$ for a duration of 3 timeslots. This setup represents a variable-duration circuit-switching schedule. However, it is possible to construct a series of matchings with fixed durations: $P_1 = \langle A \to C, B \to D, D \to A, C \to B \rangle$, $P_2 = \langle A \to C, B \to D, D \to A, C \to B \rangle$, $P_3 = \langle A \to B, B \to D, D \to C, C \to A \rangle$.

This schedule provides the desired circuit durations—a circuit between $A \rightarrow C$ for 2 timeslots and a circuit between $B \rightarrow D$ for 3 timeslots. However, this schedule is ***not* a variable-duration** schedule, as each matching requires a fixed duration. As long as each *matching* is assigned a fixed duration, the schedule is considered a fixed-duration schedule.

Definition 4 (Variable-duration circuit-switching)**.** *Any circuit-switching schedule is a sequence of matchings $P = \langle P_1, P_2, ... \rangle$. A variable-duration circuit-switching schedule specifies the execution time t_i for each matching P_i for all matchings $P_i \in P$. Importantly, the time duration for each matching is not a fixed value and can differ across matchings.*

Based on Definition 3 and Definition 4, we can formally articulate the challenge of converting BvN schedules into fixed-duration schedules. BvN schedules are inherently variable-duration schedules, as the execution time $t_i = \frac{\lambda_i}{\sum_i \lambda_i}$ for each matching P_i is not guaranteed to be uniform across all matchings.

Consider the example described above with four matchings, each requiring durations t_1, t_2, t_3, and t_4, respectively. For simplicity, assume $t_1 > t_2 > t_3 > t_4$. To convert this schedule into a fixed-duration schedule, we must select a timeslot duration Δ such that $t_1, t_2, t_3, t_4 \geq \Delta$. Suppose we choose $\Delta = t_4$. By time quantization, this would require repeating the first matching P_1 for $\frac{t_1}{t_4}$ times, the second matching P_2 for $\frac{t_2}{t_4}$ times, and so on.

The critical challenge in such quantization is that $\frac{t_1}{t_4}$ can be arbitrarily large, as the schedules generated by BvN decomposition do not guarantee a minimum value for the coefficients λ, which can lead to enormously long schedules. Unfortunately, there is no straightforward solution in the literature to quantize BvN schedules without compromising the schedule length and duty cycle. We view *Vermilion*'s matrix rounding approach as effectively mimicking such a quantization process, providing a fixed-duration schedule while achieving provably high throughput.

Q6: *What are the implementation alternatives for fast-switching fabrics?*

A6: There are two primary approaches to achieving fast optical switching [101]: fast spatial optical switches [102] and wavelength switching, that can be achieved with the combination of fast tunable lasers with wavelength selective elements like arrayed waveguide grating routers (AWGRs) [3].

Tunable lasers combined with AWGRs enable wavelength-based routing with high speed. Recent advancements in laser designs [103]—especially disaggregated architectures that separate wavelength generation from selection—have reduced tuning latencies to nanosecond levels. In addition, AWGRs are passive and robust, requiring no mechanical components or frequent upgrades. Nevertheless, the widespread adoption of this approach has been hindered by practical implementation tradeoffs. Cost-effective tunable lasers with fast tuning capabilities are not yet widely available and consume more power than their fixed-wavelength counterparts. In AWGR-based systems scalability to large ports counts is limited by the number of wavelengths available in the system.

Spatial optical switches are predominantly based on either Mach–Zehnder interferometers (MZI) or Microelectrical Mechanical Systems (MEMS) on silicon photonic platforms. Electro-optic MZI-based switches [101, 102] have been shown to achieve ns scale switching. However, they exhibit high insertion loss and increased crosstalk that limits their scalability. Silicon photonic MEMS switches [104] have been shown to achieve switching in the order of 1-10 microseconds and can reach high radix, with lower insertion loss and better crosstalk performance. Therefore, despite their somewhat slower switching speed and reliance on custom fabrication process, silicon photonic MEMS switches currently represent promising candidates for scalable fast optical switching fabrics. We leave if for future work to explore the hardware implementation of the switching fabric.

Q7: *Is it computationally feasible to derive optimal traffic-aware periodic schedules for large topologies?*

A7: The underlying problem is to construct an optimal emulated topology within a degree constraint. Appendix C presents the linear program formulation. However, in our initial experiments, the solver (Gurobi [40]) did not terminate after 30 minutes even for a 16 node topology. Our approach addresses this challenge with an novel algorithm that can quickly derive near-optimal schedules for large topologies and demand matrices.

Q8: *Given a degree constraint, what is the best topology that maximizes throughput for a given traffic matrix?*

A8: Answering this question not only allows us to then derive a periodic schedule that emulates an optimal topology but also provides insights into more constrained networks with slow reconfigurations such as those with patch-panels [32]. While some recent works focus on specific communication patterns in distributed training under a domain-specific cost model [92], the throughput-optimization problem largely remains open for general communication patterns. Our roadmap to solve this problem builds upon our observations in §2.3. Specifically, building upon Theorem 2, our main intuition is to serve the bulk portion of the traffic matrix in a traffic-aware manner using a subset of the available links within the degree constraints and tackle the residual demand with a traffic-oblivious topology, while guaranteeing high throughput. We address this question for specific degree constraints relevant for periodic networks but under general demand matrices within the hose model, providing a first step towards a general solution (see §3.2).

Q9: *Is it fundamentally feasible to achieve high throughput for any traffic matrix using only single-hop routing?*

A9: Under constrained length of the schedule (and degree), answering this question not only requires finding optimal topology under an "ideal routing" obtained from solving the concurrent flow problem, we further need to restrict the paths to direct communication. Our approach is to construct *short* periodic schedules that provide bandwidth between communicating pairs such that the gap between bandwidth and demand between any pair is bounded by a certain ratio, ensuring high throughput with single-hop routing (see §3.2).

B Throughput of traffic-aware Networks

Theorem 1 (Ideal throughput of traffic-aware network)**.** *The throughput of an ideal traffic-aware reconfigurable network is* 1 *i.e., full-throughput for any traffic matrix if the reconfiguration delay is negligible.*

Proof. Within the hose model set of demand matrices, we consider saturated demand matrices i.e., the sum of every row (column) equals the outgoing (incoming) capacity of each node. If a topology can achieve throughput θ for all saturated demand matrices, then the topology can achieve throughput θ for any traffic matrix [23]. Given that saturated demand matrices are doubly stochastic, we first decompose the matrix using Birkhoff–von Neumann (BvN) decomposition technique [18] into k permutation matrices, where k can be up to n^2. Let $\mathcal{M}$ be any saturated traffic matrix, where the sum of every row and column is $c \cdot u$ (total capacity of each node). Let the corresponding BvN decomposition be $\mathcal{M} = \lambda_1 \cdot P_1 + \lambda_2 \cdot P_2 ... + \lambda_k P_k$, where P_i is a permutation matrix and the coefficients λ are such that $\sum_{i=1}^{k} \lambda = c \cdot u$. Using this decomposition, we configure the topology such that each permutation P_i is executed using full node capacity $c \cdot u$ for $\frac{\lambda_i}{c \cdot u} \cdot \Delta$ units of time over a period of one unit of time Δ. Over Δ amount of time, $\lambda_i \cdot P_i$ portion of the traffic matrix generates $\lambda_i \cdot P_i \cdot \Delta$ demand in volume. As a result, during $\frac{\lambda_i}{c \cdot u} \cdot \Delta$ amount of time, by executing the corresponding permutation P_i using full capacity $c \cdot u$, the topology can fully satisfy $\lambda_i \cdot P_i$ portion of the traffic matrix. As a result, the topology can fully satisfy the traffic matrix $\mathcal{M}$ over each period of one unit of time Δ and achieves full throughput. □

Theorem 2 (Throughput under integer traffic matrices)**.** *There exists a periodic reconfigurable network with a period of* Γ *timeslots with* Δ_r *fraction of time spent in reconfiguration, that can achieve nearly full throughput of* $1 - \Delta_r$ *using only single-hop routing for traffic matrices where any non-zero demand is an integer multiple of* $c \cdot \frac{1}{\Gamma}$*, where* c *is the link capacity in the physical topology.*

Proof. A traffic matrix $\mathcal{M}$ in the hose model has the property that the sum of every row and column is at most $\hat{d}$, where $\hat{d}$ is the degree (physical links). Further, we have that every entry in the matrix is an integer multiple of $c \cdot \frac{1}{\Gamma}$, where c is the link capacity and Γ is the period of the periodic schedule. We assume, without loss of generality, that the capacity of each physical link is 1. The emulated multigraph G has a degree of $\Gamma \cdot \hat{d}$, and each link has capacity $\frac{1}{\Gamma}$. We now upscale both the traffic matrix $\mathcal{M}$ and each link capacity of the emulated multigraph by Γ. Thus, it is equivalent to find the throughput of the emulated multigraph G with degree $\Gamma \cdot \hat{d}$ and each link having capacity 1, under the scaled traffic matrix $\mathcal{M}' = \Gamma \cdot \mathcal{M}$, where the sum of each row and column is at most $\Gamma \cdot \hat{d}$. Since every entry in the matrix $\mathcal{M}$ is an integer multiple of $\frac{1}{\Gamma}$, the scaled traffic matrix $\mathcal{M}'$ is an integer matrix with sum of every row and column at most $\Gamma \cdot \hat{d}$. Further, we have a degree of $\Gamma \cdot \hat{d}$ with link capacity of 1. It is now straight-forward that constructing a graph by adding links between each pair based on the value of the demand in the scaled matrix $\mathcal{M}'$ can fully satisfy the demand, and this requires only single-hop routing. □

C Linear Program Formulation

We emphasize that the following linear program formulation ***is not related to*** *Vermilion's* ***design***. Instead, the goal of this section is to formulate the underlying problem of finding the optimal emulated graph for a given traffic matrix under a degree constraint. This formulation serves as a theoretical exercise to understand the complexity of the problem, which motivates the development of efficient algorithms like *Vermilion*.

Throughput maximization is a variant of multi-commodity maximum flow problem, commonly referred as maximum concurrent flow problem [26]. In the case of traffic-aware periodic networks, our goal is to find the best emulated graph. In the following we present an integer linear program formulation. Given a network of n nodes, each with $\hat{d}$ physical links (incoming and outgoing), the LP takes traffic matrix $\mathcal{M}$ and the length of the desired schedule Γ as input. The capacity of the physical links is denoted by c. The LP has to find the number of links $\hat{c}^{i,j}$ between each node pair (i,j). We set the link capacities to 1 and interpret $\hat{c}^{i,j}$ as the capacity between i, j. We use $f_{i,j}^{s,d}$ to refer to the flow on edge (i,j) corresponding to (s,d) demand. Our objective is to maximize throughput θ such that the scaled traffic matrix $\theta \cdot \mathcal{M}$ satisfies source-destination demands, flow conservation and capacity constraints. We consider a degree constraint $\hat{d}$ for each node. Consequently, the demand matrices of interest are those with the sum of each row and column limited to $\hat{d}$.

Input:

$$\begin{aligned} &\text{traffic matrix} && \mathcal{M} = \{m_{s,d} \mid s \in V, d \in V\} \\ &\text{In-out degree} && \hat{d} \end{aligned}$$

Objective Function:

$$\text{Maximize} \quad \theta$$

Subject to the constraints:

$$\text{Source demand: } \sum_{i \in V \setminus \{s\}} f_{s,i}^{s,d} \geq \theta \cdot m_{s,d} \qquad \forall s \in V, \forall d \in V$$

$$\text{Destination demand: } \sum_{i \in V \setminus \{d\}} f_{i,d}^{s,d} \geq \theta \cdot m_{s,d} \qquad \forall s \in V, \forall d \in V$$

$$\text{Flow conservation: } \sum_{i \in V \setminus \{j\}} f_{i,j}^{s,d} - \sum_{k \in V \setminus \{j\}} f_{j,k}^{s,d} = 0 \qquad \forall j \in V \setminus \{s,d\} \quad \forall s \in V, \forall d \in V$$

$$\text{Capacity constraints: } \sum_{s \in V} \sum_{d \in V} f_{i,j}^{s,d} \leq \hat{c}^{i,j} \qquad \forall i \in V, \forall j \in V$$

$$\text{In-degree constraints: } \sum_{s \in V} c^{s,d} \leq \hat{d} \qquad \forall d \in V$$

$$\text{Out-degree constraints: } \sum_{s \in V} c^{s,d} \leq \hat{d} \qquad \forall d \in V$$

Variables:

$$\text{Flow: } f_{i,j}^{s,d} \geq 0 \,,\, f_{i,j}^{s,d} \in \mathbb{R} \qquad \forall i \in V, j \in V, s \in V, \forall d \in V$$

$$\text{Throughput: } \theta \geq 0 \,,\, \theta \in \mathbb{R}$$

$$\text{Number of links: } \hat{c}^{i,j} \geq 0 \,,\, \hat{c}^{i,j} \in \mathbb{Z}$$

D Limitations and Future Work

Vermilion represents an initial step toward traffic-aware periodic networks capable of achieving high throughput for any traffic matrix using only single-hop routing. However, several challenges remain and open avenues for future research.

Temporal dependencies in communication patterns: *Vermilion* assumes that the traffic matrix, whether defined by rate (bits per second) or volume (bits), is available and that the demands between source-destination pairs are independent. However, certain workloads, such as distributed training, exhibit temporal dependencies in their communication patterns. For example, while the traffic matrix may accurately represent traffic, specific portions of the demand (e.g., from the backward pass) may only become available after the completion of other parts (e.g., from the forward pass) [33, 34, 105]. Addressing these temporal dependencies presents a significant research challenge and offers opportunities to further optimize topologies like *Vermilion* for such workloads in a fully optical setting. Hybrid network design with dedicated electrical packet switches (e.g., Mixnet [22]) offer a potential solution and a transitional step towards fully optical traffic-aware periodic networks, which we leave for future work.

Structured communication patterns: *Vermilion* is designed to achieve high throughput for any arbitrary traffic matrix. However, some communication patterns have inherent structure that could be exploited for further optimization. For example, the ring-allReduce collective communication, commonly used in distributed training, can be efficiently supported by a simple ring-emulated topology with periodic schedules. Optimizing for specific communication patterns is complementary to our approach. In principle, *Vermilion* could be extended to recognize and leverage these structured communication patterns using existing solutions [92], potentially achieving even higher throughput and faster completion times. Exploring these optimizations is an avenue for future work.

Fault-tolerance and resilience: *Vermilion* does not explicitly address fault-tolerance or resilience. While the periodic nature of the network may offer some inherent resilience to failures, designing fault-tolerant periodic schedules remains an open challenge. Failures in optical networks can be particularly difficult to detect, as they often manifest as packet corruption due to optical collisions. Recent work discusses techniques to mitigate link-layer and physical-layer errors [16]. Future work could explore the design of fault-tolerant schedules that can quickly adapt to failures while still maintaining high throughput.

Heterogeneous link capacities: In this paper, we assume that all physical links in the topology have uniform capacity. However, datacenter topologies often include links with heterogeneous capacities. For example, faulty auto-negotiation between two NICs can result in a link operating at a lower capacity than expected. In practice, these capacities are often multiples of a base rate. We believe *Vermilion* can be generalized to accommodate heterogeneous link capacities by selecting an appropriate base capacity and adjusting the topology to handle varying multiples of that capacity. We leave the generalization of *Vermilion* for heterogeneous link capacities to future work.

E Additional Results

Figure 11 presents throughput results for a network with 48 nodes, which exhibit trends consistent with those observed for the 16-node network in §4.3. Figure 12 shows the flow completion times (FCT) achieved by *Vermilion* and the baselines for the web search workload under varying load conditions. *Vermilion* significantly improves the 99th percentile FCTs compared to existing designs, for both short and long flows.

Figures 13, 14, and 15 report the all-to-all completion time speedup, defined as $\frac{\text{Alg}}{\textit{Vermilion}}$, for Qwen-MoE, Mixtral-8$\times$7B, and Mixtral-8$\times$22B workloads across a range of reconfiguration delays and optical fabric link bandwidths. Compared to MixNet [22], *Vermilion* achieves substantial speedups in

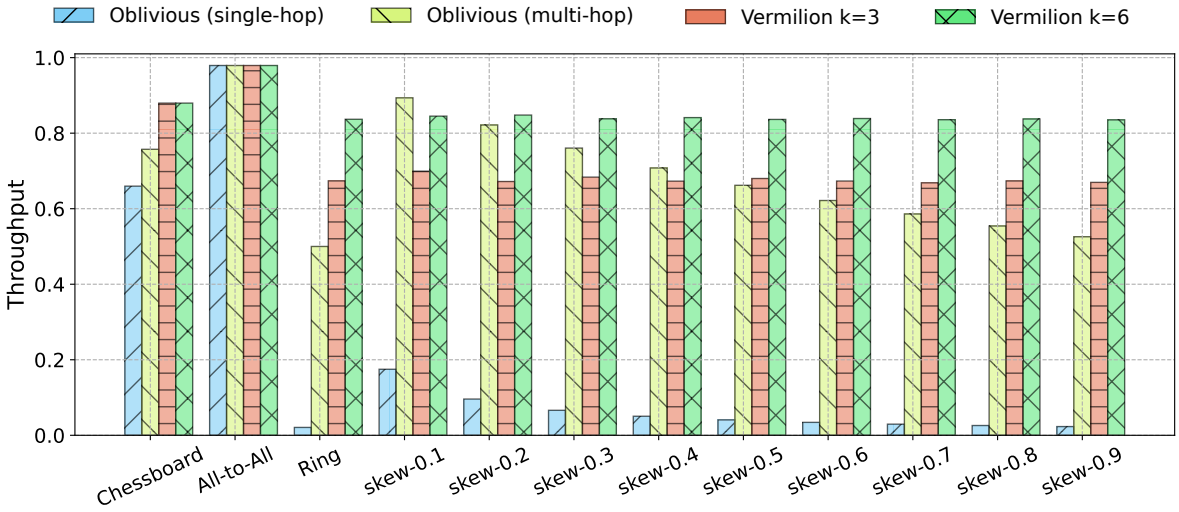
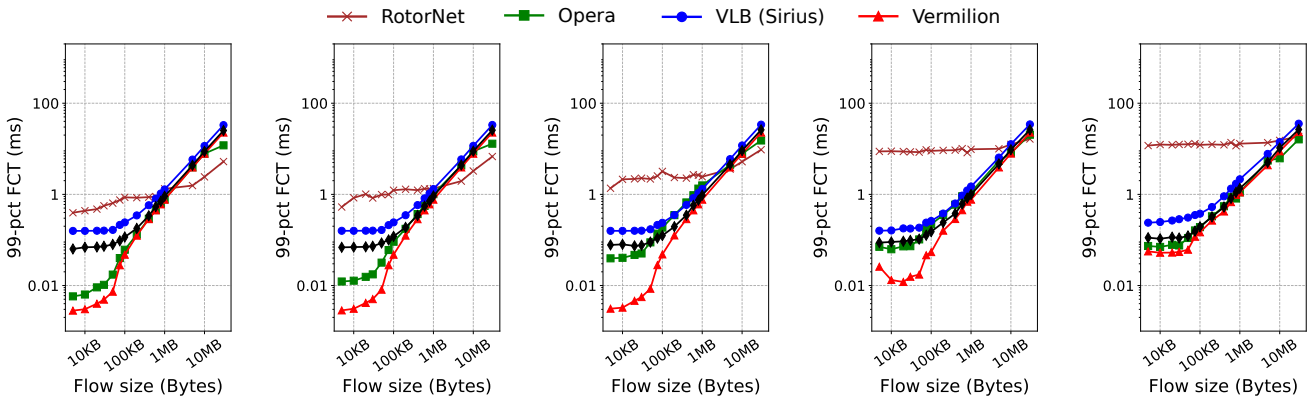


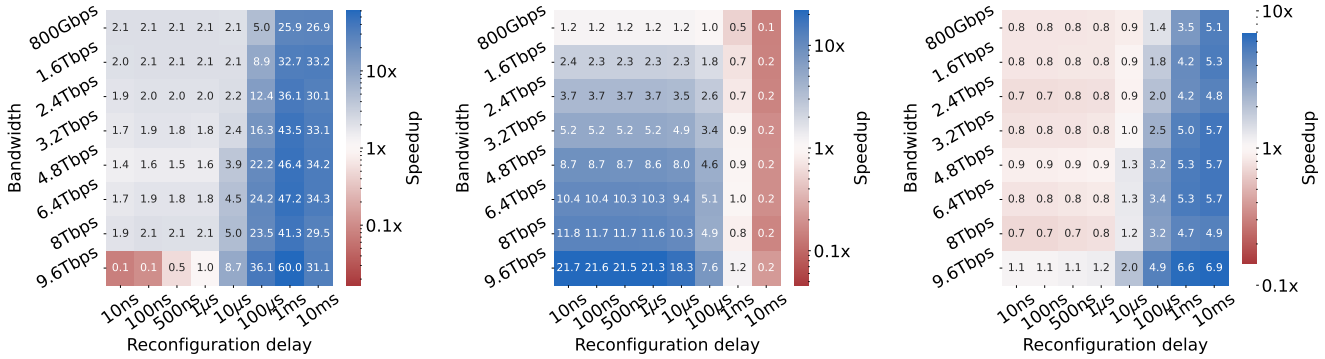
Figure 11: Throughput of oblivious periodic network and *Vermilion* for a 48 node topology. *Vermilion* achieves higher throughput compared to oblivious networks, even with single-hop direct communication.



(a) Load=5% (b) Load=10% (c) Load=20% (d) Load=40% (e) Load=60%

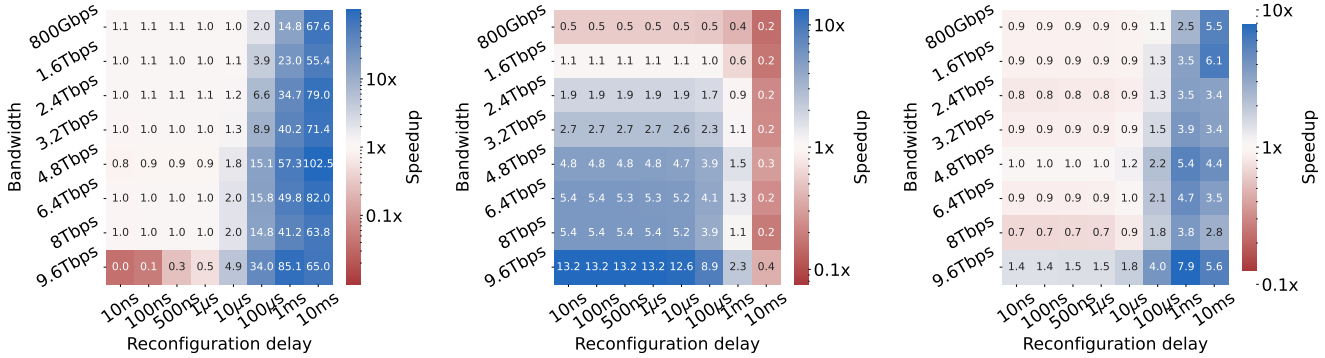
Figure 12: Flow completion times for the websearch workload. *Vermilion* significantly improves the 99-percentile FCTs compared to existing designs, both for short flows and long flows.

regimes with high optical bandwidth and moderate reconfiguration delays (e.g., 10 μs), where periodic networks are expected to perform well. Compared to oblivious periodic switching (VLB) and Birkhoff–von Neumann (BvN) schedules, *Vermilion* consistently achieves higher speedups as reconfiguration delays increase, due to its ability to sustain high throughput while maintaining short schedules, unlike BvN.



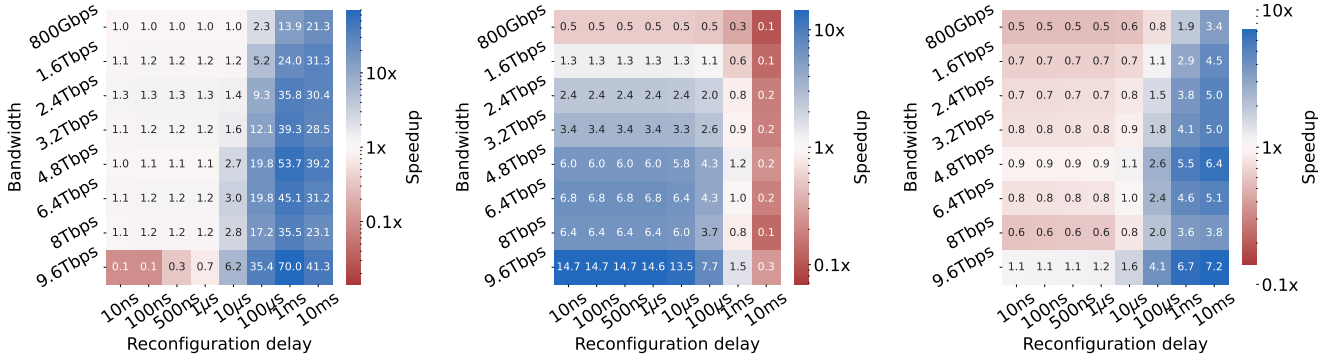
(a) VLB (b) MixNet (c) BvN

Figure 13: All-to-All completion time speedup $\frac{Alg}{Vermilion}$ for Qwen MoE workload across different reconfiguration delays and optical fabric link bandwidths.



(a) VLB (b) MixNet (c) BvN

Figure 14: All-to-All completion time speedup $\frac{Alg}{Vermilion}$ for Mixtral-8x7B workload across different reconfiguration delays and optical fabric link bandwidths.



(a) VLB (b) MixNet (c) BvN

Figure 15: All-to-All completion time speedup $\frac{Alg}{Vermilion}$ for Mixtral-8x22B workload across different reconfiguration delays and optical fabric link bandwidths.